

National University of Singapore

Just lean on the meme

Nguyen Huu Nhat Quang, Le Pham Duy Khoa, Thieu Nguyen Huy

2026-11-09

Contents

Contest (1)

template.cpp

```cpp
#include <bits/stdc++.h>
#include <cassert>
using namespace std;
#define rep(i, a, b) for(int i = a; i < (b); ++i)
#define all(x) begin(x), end(x)
#define sz(x) (int)(x).size()
typedef long long ll;
typedef pair<int, int> pii;
typedef vector<int> vi;
int main() {
  cin.tie(0)->sync_with_stdio(0);
  cin.exceptions(cin.failbit);
}
```

.bashrc

```
alias c='g++ -Wall -Wconversion -Wfatal-errors -g -std=c++17 \
  -fsanitize=undefined,address'
xmodmap -e 'clear lock' -e 'keycode 66=less greater' # caps = <>
```

hash.sh

```
# Hashes a file, ignoring all whitespace and comments. Use for
# verifying that code was correctly typed.
cpp -dD -P -fpreprocessed | tr -d '[:space:]'| md5sum | cut -c-6
```

debug.sh

```
set -e
g++ code.cpp -o code
g++ gen.cpp -o gen
g++ brute.cpp -o brute
for((i = 1; ; ++i)); do
    ./gen $i > input_file
    ./code < input_file > myAnswer
    ./brute < input_file > correctAnswer
    diff -Z myAnswer correctAnswer > /dev/null || break
    echo "Passed test: "  $i
done
echo "WA on the following test:"
cat input_file
echo "Your answer is:"
cat myAnswer
echo "Correct answer is:"
cat correctAnswer
```

pragma.h

```
#pragma GCC optimize("O3,Ofast,unroll-loops")
#pragma GCC target("avx2,bmi,bmi2,lzcnt,popcnt")
```

troubleshoot.txt

```
Set and multiset
```

Mathematics (2)

2.1 Fibonacci Sequence

Denote sequence F_n as: $F_0 = 0, F_1 = 1$,
$F_n = F_{n-1} + F_{n-2} \quad \forall n \geq 2$. Some properties below:

- $F_n = \dfrac{\phi^n - \psi^n}{\sqrt{5}}$, where $\phi = \frac{1+\sqrt{5}}{2}$, $\psi = \frac{1-\sqrt{5}}{2}$
- $F_{n-1}F_{n+1} - F_n^2 = (-1)^n$.
- $F_{n+k} = F_k F_{n+1} + F_{k-1}F_n$. For $k = n$: $F_{2n} = F_n(F_{n+1} + F_{n-1})$
- $F_n \mid F_m \leftrightarrow n \mid m \quad \forall n \geq 3$. , $\gcd(F_m, F_n) = F_{\gcd(m,n)}$.
- F_{n+2} equals no. of ways to choose zero or more positions in $\{1, 2, \ldots, n\}$ s/t no two chosen positions are consecutive.

2.2 Sums

- $1^2 + 2^2 + \ldots + n^2 = \dfrac{n(n+1)(2n+1)}{6}$.
- $1^3 + 2^3 + \ldots + n^4 = \dfrac{n^2(n+1)^2}{4} = (1 + 2 + \ldots + n)^2$
- $1^4 + 2^4 + \cdots + n^4 = \dfrac{n(n+1)(2n+1)(3n^2+3n-1)}{30}$.

2.3 Mobius

Define $s_f(n) = \sum_{i=1}^{n} f(i)$ and $(f * g)(n) = \sum_{d|x} f(d)g(\frac{x}{d})$
(Dirichlet Convolution). Then, the following holds:

- $s_{f*g}(n) = \sum_{i=1}^{n} \sum_{d|i} g(d) f\left(\frac{i}{d}\right) = \sum_{d=1}^{n} \sum_{p=1}^{\lfloor \frac{n}{d} \rfloor} g(d) f(p)$
- $s_{f*g}(n) = g(1) \sum_{p=1}^{n} f(p) + \sum_{d=2}^{n} \sum_{p=1}^{\lfloor \frac{n}{d} \rfloor} g(d) f(p)$
- $s_{f*g}(n) = g(1) s_f(n) + \sum_{d=2}^{n} g(d) s_f\left(\left\lfloor \frac{n}{d} \right\rfloor\right)$
- $s_f(n) = \dfrac{s_{f*g}(n) - \sum_{d=2}^{n} s_f\left(\left\lfloor \frac{n}{d} \right\rfloor\right) g(d)}{g(1)}$

2.4 Useful Triangle Formulas

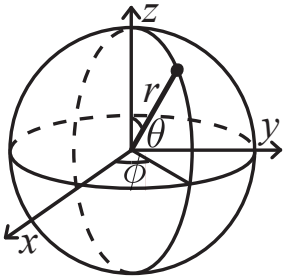
For a triangle made up of three points A, B, C, denote:

- $a = |AB|, b = |BC|, c = |CA|$, $s = \dfrac{a+b+c}{2}$
- $\angle A = \angle BAC, \angle B = \angle ABC, \angle C = \angle ACB$
- R as the radius of the circumcircle.
- r as the radius of the incircle.
- A as the area of the triangle.

Then, some useful formulas are:

- $A = \sqrt{s(s-a)(s-b)(s-c)} = rs = \dfrac{abc}{4R}$
- $2A = ab\sin(\angle A) = bc\sin(\angle B) = ca\sin(\angle C)$
- $\dfrac{a}{\sin(\angle A)} = \dfrac{b}{\sin(\angle B)} = \dfrac{c}{\sin(\angle C)} = 2R$.
- $a^2 = b^2 + c^2 - 2bc\cos(\angle A), b^2 = c^2 + a^2 - 2ca\cos(\angle B)$, $c^2 = a^2 + b^2 - 2ab\cos(\angle C)$

2.5 Spherical Coordinates



Combinatorial (3)

Note: For convention, denote C_n^k as n choose k.

3.1 Permutations

3.1.1 Factorial

n	1	2	3	4	5	6	7	8	9	10
$n!$	1	2	6	24	120	720	5040	40320	362880	3628800

n	11	12	13	14	15	16	17
$n!$	4.0e7	4.8e8	6.2e9	8.7e10	1.3e12	2.1e13	3.6e14

n	20	25	30	40	50	100	150	171
$n!$	2e18	2e25	3e32	8e47	3e64	9e157	6e262	>DBL_MAX

3.1.2 Cycles

$g_S(n)$ is the no. of permutations of $p_1, \ldots, p_n$ s/t their cycle lengths on the permutation graph $p_i - i$ all belong to the set S. Then we have the formula: $\sum_{n=0}^{\infty} g_S(n)\frac{x^n}{n!} = \exp\left(\sum_{n\in S}\frac{x^n}{n}\right)$

3.1.3 Derangements

$f(n)$ is no. of permutations $p_1, \ldots, p_n$ s/t $p_i \neq i \quad \forall i \in [1, n]$. Base cases are $f(0) = 1, f(1) = 0$. Formulas below:

- $f(n) = (n-1)(f(n-1) + f(n-2)) \quad \forall n \geq 2$.
- $f(n) = nf(n-1) + (-1)^n \quad \forall n \geq 1$.
- $f(n) = n!\sum_{k=0}^{n}\frac{(-1)^k}{k!} \quad \forall n \geq 0$.

3.1.4 Burnside's lemma

Given a set G of functions on a set X satisfying:

- $\forall g \in G$, $g(x)$ is a bijection $X \to X$.
- For $g_1, g_2 \in G$, then $g_1 \circ g_2$ (shorthand g_1g_2) $\in G$.
- $\exists e \in G$ s/t $e(x) = x \quad \forall x \in X$.

Then, no. of equivalence classes on G is:
$|X/R| = \frac{1}{|G|}\sum_{g\in G}|X^g|$ where $X^g = \{x \in X : g(x) = x\}$.

If $f(n)$ counts "configurations" (of some sort) of length n, we can ignore rotational symmetry using G as the set of all cyclic shifts: Ans $= \frac{1}{n}\sum_{k=0}^{n-1} f(\gcd(n,k)) = \frac{1}{n}\sum_{k|n} f(k)\phi(n/k)$.

3.2 Partitions and subsets

3.2.1 Binomial Coefficient

Some nice properties of C_n^k:

- $C_n^k = C_n^{n-k} = \frac{n}{k}\ C_{k-1}^{n-1}, \sum_{k=0}^{n} C_n^k = 2^n$.
- $\sum_{m=0}^{n} C_m^k = C_{k+1}^{n+1}, \sum_{k=0}^{m} C_{n+k}^k = C_{n+m+1}^m$
- $\sum_{k=0}^{n}(C_n^k)^2 = C_{2n}^n$. , $\sum_{k=0}^{n} k \cdot C_n^k = n2^{n-1}$
- $\sum_{k=0}^{n} C_{n-k}^k = F_{n+1}$ (Fibonacci number).
- $\sum_{k=0}^{x}\frac{C_x^k}{C_n^k} = \frac{n+1}{n-x+1}$.

3.2.2 Partition function

Denote $p(n)$ as no. of ways of writing n as a sum of positive integers, disregarding the order of the summands. Formulas below:

$$p(0) = 1,\ p(n) = \sum_{k\in\mathbb{Z}\setminus\{0\}} (-1)^{k+1} p(n - k(3k-1)/2)$$

Approximation: $p(n) \sim 0.145/n \cdot \exp(2.56\sqrt{n})$.:

n	0	1	2	3	4	5	6	7	8	9	20	50	100
$p(n)$	1	1	2	3	5	7	11	15	22	30	627	~2e5	~2e8

3.2.3 Lucas' Theorem

Let n, m be non-negative integers and p a prime. Write $n = n_k p^k + ... + n_1 p + n_0$ and $m = m_k p^k + ... + m_1 p + m_0$. Then $C_n^m \equiv \prod_{i=0}^{k} C_{n_i}^{m_i} \pmod p$.

3.3 General purpose numbers

3.3.1 Bernoulli numbers

EGF of Bernoulli numbers is $B(t) = \frac{t}{e^t - 1}$ (FFT-able).
$B[0, \ldots] = [1, -\frac{1}{2}, \frac{1}{6}, 0, -\frac{1}{30}, 0, \frac{1}{42}, \ldots]$

Sums of powers:

$$\sum_{i=1}^{n} i^m = \frac{1}{m+1}\sum_{k=0}^{m}\binom{m+1}{k} B_k \cdot (n+1)^{m+1-k}$$

Euler-Maclaurin formula for infinite sums:

$$\sum_{i=m}^{\infty} f(i) = \int_m^{\infty} f(x)dx - \sum_{k=1}^{\infty}\frac{B_k}{k!} f^{(k-1)}(m)$$

$$\approx \int_m^{\infty} f(x)dx + \frac{f(m)}{2} - \frac{f'(m)}{12} + \frac{f'''(m)}{720} + O(f^{(5)}(m))$$

3.3.2 Stirling numbers of the first kind

Denote $c(n, k)$ as no. of permutations length n with k cycles on the permutation graph $p_i - i$. Formulas below:

- $c(n, k) = c(n-1, k-1) + (n-1)c(n-1, k),\ c(0, 0) = 1$
- $\sum_{k=0}^{n} c(n, k)x^k = x(x+1)\ldots(x+n-1)$

$c(8, k) = 0, 5040, 13068, 13132, 6769, 1960, 322, 28, 1$

$c(n, 2) = 0, 0, 1, 3, 11, 50, 274, 1764, 13068, 109584, \ldots$

3.3.3 Eulerian numbers

Denote $E(n, k)$ as no. of permutations $p_1, \ldots, p_n$ s/t exactly k positions i satisfies $i \geq 2, p_i > p_{i-1}$. Formulas below:

- Base cases: $E(n, 0) = E(n, n-1) = 1$
- $E(n, k) = (n-k)E(n-1, k-1) + (k+1)E(n-1, k)$
- $E(n, k) = \sum_{j=0}^{k}(-1)^j\binom{n+1}{j}(k+1-j)^n$

3.3.4 Stirling number of the second kind

Denote $S(n, k)$ as no. of ways to partition the set $1, 2, \ldots, n$ into k non-empty, unlabeled subsets. Formulas below:

- Base cases: $S(0, 0) = 1, S(n, 0) = 0 \quad \forall n > 0$.
- $S(n, k) = k \cdot S(n-1, k) + S(n-1, k-1) \quad \forall k > 0$.
- $S(n, k) = \frac{1}{k!}\sum_{i=0}^{k}(-1)^{k-i} \cdot C_k^i \cdot i^n$.

3.3.5 Bell numbers

Denote $B(n)$ as no. of partitions of n distinct elements.
$B(0), B(1), \ldots = 1, 1, 2, 5, 15, 52, 203, 877, 4140, 21147, \ldots$

For p prime, $B(p^m + n) \equiv mB(n) + B(n+1) \pmod p$

3.3.6 Prufer code

Prufer code is a bijection f from an unrooted, labeled tree with n vertices ($n \geq 2$) to an array of $n-2$ integers, where each integer is in the range $[0, n-1]$.

For a tree T with degrees $d_0, \ldots, d_{n-1}$, the number of times x appears in $f(T)$ is exactly $d_x - 1\ \forall x \in [0, n-1]$.

Some nice formulas derived from Prufer Code: (Assume all trees are labeled and unrooted)

- No. of trees of n vertices is n^{n-2}
- If there are k trees, each with $n_1, \ldots, n_k$ vertices then no. of trees with $n = n_1 + \ldots + n_k$ made by connecting the k trees is $n_1 n_2 \ldots n_k n^{k-2}$.
- No. of trees of n vertices s/t it has degrees $d_0, \ldots, d_{n-1}$ is $\dfrac{(n-2)!}{(d_0-1)! \ldots (d_{n-1}-1)!}$

3.3.7 Catalan Numbers

Denote C_n as these equivalent definitions:

- No. of regular bracket sequences of length $2n$,
- No. of rooted unlabeled binary trees with $n+1$ leaves,
- No. of paths from $(0,0)$ to (n,n) that never goes above the line $x = y$.
- No. of ways to triangulate a convex polygon with $n+2$ sides.
- No. of binary search trees that can be constructed with n distinct keys.
- No. of unlabeled, ordered trees with $n+1$ vertices.
- No. of permutations of length n such that the LIS is at most 2.

Formulas to calculate C_n:

- $C_n = \dfrac{1}{n+1} C(n, 2n) \quad \forall n \geq 0.$
- $C_n = C(n, 2n) - C(n+1, 2n) \quad \forall n \geq 0.$
- $C_{n+1} = \sum_{i=0}^{n} C_i C_{n-i} \quad \forall n \geq 0.$

$C_0, C_1, \ldots$ is: $1, 1, 2, 5, 14, 42, 132, 429, 1430, 4862, \ldots$

Number theory (4)

4.1 Modular arithmetic

ModularArithmetic.h

Description: Operators for modular arithmetic. You need to set `mod` to some number first and then you can use the structure.

"euclid.h" 35bfea, 18 lines

```
const ll mod = 17; // change to something else
struct Mod {
  ll x;
  Mod(ll xx) : x(xx) {}
  Mod operator+(Mod b) { return Mod((x + b.x) % mod); }
  Mod operator-(Mod b) { return Mod((x - b.x + mod) %
      mod); }
  Mod operator*(Mod b) { return Mod((x * b.x) % mod); }
  Mod operator/(Mod b) { return *this * invert(b); }
  Mod invert(Mod a) {
    ll x, y, g = euclid(a.x, mod, x, y);
    assert(g == 1); return Mod((x + mod) % mod);
  }
  Mod operator^(ll e) {
    if (!e) return Mod(1);
    Mod r = *this ^ (e / 2); r = r * r;
    return e&1 ? *this * r : r;
  }
};
```

ModInverse.h

Description: Pre-computation of modular inverses. Assumes LIM $\leq$ mod and that mod is a prime.

Status: Works

db454d, 3 lines

```
const ll mod = 1000000007, LIM = 200000;
vector<ll> inv(LIM); inv[1] = 1;
rep(i,2,LIM) inv[i] = mod - (mod / i) * inv[mod % i] %
    mod;
```

ModSqrt.h

Description: Tonelli-Shanks algorithm for modular square roots. Finds x s.t. $x^2 = a \pmod p$ ($-x$ gives the other solution).

Time: $\mathcal{O}\left(\log^2 p\right)$ worst case, $\mathcal{O}\left(\log p\right)$ for most p

Status: Tested for all a,p <= 10000

19dd12, 33 lines

```
namespace mod_sqrt_detail {
ll power(ll a, ll e, ll p) {
  ll res = 1;
  for (; e; e >>= 1, a = a * a % p)
    if (e & 1) res = res * a % p;
  return res;
}
}
ll sqrt(ll a, ll p) {
  a %= p; if (a < 0) a += p;
  if (p == 2) return a;
  if (a == 0) return 0;
  assert(mod_sqrt_detail::power(a, (p-1)/2, p) == 1); //
      else no solution
  if (p % 4 == 3) return mod_sqrt_detail::power(a, (p+1)
      /4, p);
  // a^(n+3)/8 or 2^(n+3)/8 * 2^(n-1)/4 works if p % 8
      == 5
  ll s = p - 1, n = 2;
  int r = 0, m;
  while (s % 2 == 0)
    ++r, s /= 2;
  while (mod_sqrt_detail::power(n, (p - 1) / 2, p) != p
      - 1) ++n;
  ll x = mod_sqrt_detail::power(a, (s + 1) / 2, p);
  ll b = mod_sqrt_detail::power(a, s, p), g =
      mod_sqrt_detail::power(n, s, p);
  for (;; r = m) {
    ll t = b;
    for (m = 0; m < r && t != 1; ++m)
      t = t * t % p;
    if (m == 0) return x;
    ll gs = mod_sqrt_detail::power(g, 1LL << (r - m - 1)
        , p);
    g = gs * gs % p;
    x = x * gs % p;
    b = b * g % p;
  }
}
```

4.2 Primality

FastEratosthenes.h

Description: Prime sieve for generating all primes smaller than LIM.

Time: LIM=1e9 $\approx$ 1.5s

Status: Stress-tested

6b2912, 20 lines

```
const int LIM = 1e6;
bitset<LIM> isPrime;
vi eratosthenes() {
  const int S = (int)round(sqrt(LIM)), R = LIM / 2;
  vi pr = {2}, sieve(S+1); pr.reserve(int(LIM/log(LIM)
      *1.1));
  vector<pii> cp;
  for (int i = 3; i <= S; i += 2) if (!sieve[i]) {
    cp.push_back({i, i * i / 2});
    for (int j = i * i; j <= S; j += 2 * i) sieve[j] =
        1;
  }
  for (int L = 1; L <= R; L += S) {
    array<bool, S> block{};
    for (auto &[p, idx] : cp)
      for (int i=idx; i < S+L; idx = (i+=p)) block[i-L]
          = 1;
    rep(i,0,min(S, R - L))
      if (!block[i]) pr.push_back((L + i) * 2 + 1);
  }
  for (int i : pr) isPrime[i] = 1;
  return pr;
}
```

MillerRabin.h

Description: Deterministic Miller-Rabin primality test. Guaranteed to work for numbers up to $7 \cdot 10^{18}$; for larger numbers, use Python and extend A randomly.

Time: 7 times the complexity of $a^b \mod c$.

Status: Stress-tested

"ModMul.h" 60dcd1, 12 lines

```
bool isPrime(ull n) {
  if (n < 2 || n % 6 % 4 != 1) return (n | 1) == 3;
```

```
  ull A[] = {2, 325, 9375, 28178, 450775, 9780504,
      1795265022},
    s = __builtin_ctzll(n-1), d = n >> s;
  for (ull a : A) {   // ^ count trailing zeroes
    ull p = modpow(a%n, d, n), i = s;
    while (p != 1 && p != n - 1 && a % n && i--)
      p = modmul(p, p, n);
    if (p != n-1 && i != s) return 0;
  }
  return 1;
}
```

PollardRho.h

Description: Pollard-rho randomized factorization algorithm. Returns prime factors of a number, in arbitrary order (e.g. 2299 -> {11, 19, 11}).
Time: $\mathcal{O}\left(n^{1/4}\right)$, less for numbers with small factors.
Status: stress-tested

"ModMul.h", "MillerRabin.h" d8d98d, 18 lines

```
ull pollard(ull n) {
  ull x = 0, y = 0, t = 30, prd = 2, i = 1, q;
  auto f = [&](ull x) { return modmul(x, x, n) + i; };
  while (t++ % 40 || __gcd(prd, n) == 1) {
    if (x == y) x = ++i, y = f(x);
    if ((q = modmul(prd, max(x,y) - min(x,y), n))) prd =
        q;
    x = f(x), y = f(f(y));
  }
  return __gcd(prd, n);
}
vector<ull> factor(ull n) {
  if (n == 1) return {};
  if (isPrime(n)) return {n};
  ull x = pollard(n);
  auto l = factor(x), r = factor(n / x);
  l.insert(l.end(), all(r));
  return l;
}
```

4.3 Divisibility

euclid.h

Description: Finds two integers x and y, such that $ax + by = \gcd(a, b)$. If you just need gcd, use the built in __gcd instead. If a and b are coprime, then x is the inverse of $a \pmod b$.

33ba8f, 5 lines

```
ll euclid(ll a, ll b, ll &x, ll &y) {
  if (!b) return x = 1, y = 0, a;
  ll d = euclid(b, a % b, y, x);
  return y -= a/b * x, d;
}
```

Euclid.java

Description: Finds {x, y, d} s.t. ax + by = d = gcd(a, b).
Status: Tested

6aba01, 11 lines

```
static BigInteger[] euclid(BigInteger a, BigInteger b) {
  BigInteger x = BigInteger.ONE, yy = x;
  BigInteger y = BigInteger.ZERO, xx = y;
  while (b.signum() != 0) {
    BigInteger q = a.divide(b), t = b;
    b = a.mod(b); a = t;
    t = xx; xx = x.subtract(q.multiply(xx)); x = t;
    t = yy; yy = y.subtract(q.multiply(yy)); y = t;
  }
  return new BigInteger[]{x, y, a};
}
```

CRT.h

Description: Chinese Remainder Theorem.
crt(a, m, b, n) computes x such that $x \equiv a \pmod m$, $x \equiv b \pmod n$. If $|a| < m$ and $|b| < n$, x will obey $0 \le x < \text{lcm}(m, n)$. Assumes $mn < 2^{62}$.
Time: $\log(n)$
Status: Works

"euclid.h" 04d93a, 7 lines

```
ll crt(ll a, ll m, ll b, ll n) {
  if (n > m) swap(a, b), swap(m, n);
  ll x, y, g = euclid(m, n, x, y);
  assert((a - b) % g == 0); // else no solution
  x = (b - a) % n * x % n / g * m + a;
  return x < 0 ? x + m*n/g : x;
}
```

4.3.1 Bézout's identity

For $a \neq 0$, $b \neq 0$, then $d = gcd(a, b)$ is the smallest positive integer for which there are integer solutions to

$$ax + by = d$$

If (x, y) is one solution, then all solutions are given by

$$\left(x + \frac{kb}{\gcd(a,b)}, y - \frac{ka}{\gcd(a,b)}\right), \quad k \in \mathbb{Z}$$

phiFunction.h

Description: $\phi(1) = 1$. If $n = p_1^{k_1} p_2^{k_2} ... p_r^{k_r}$ then $\phi(n) = (p_1 - 1)p_1^{k_1 - 1} ... (p_r - 1)p_r^{k_r - 1}$. $\phi(n) = n \cdot \prod_{p|n}(1 - 1/p)$.
$\sum_{d|n} \phi(d) = n$, $\sum_{1 \le k \le n, \gcd(k,n)=1} k = n\phi(n)/2, n > 1$
Euler's thm: a, n coprime $\Rightarrow a^{\phi(n)} \equiv 1 \pmod n$.
Status: Tested

cf7d6d, 7 lines

```
const int LIM = 5000000;
int phi[LIM];
void calculatePhi() {
  rep(i,0,LIM) phi[i] = i&1 ? i : i/2;
  for (int i = 3; i < LIM; i += 2) if(phi[i] == i)
    for (int j = i; j < LIM; j += i) phi[j] -= phi[j] /
        i;
}
```

4.4 Fractions

ContinuedFractions.h

Description: Given N and a real number $x \ge 0$, finds the closest rational approximation p/q with $p, q \le N$. It will obey $|p/q - x| \le 1/qN$.
For consecutive convergents, $p_{k+1}q_k - q_{k+1}p_k = (-1)^k$. $(p_k/q_k$ alternates between $> x$ and $< x$.) If x is rational, y eventually becomes ∞; if x is the root of a degree 2 polynomial the a's eventually become cyclic.
Time: $\mathcal{O}(\log N)$
Status: stress-tested for n <= 300

dd6c5e, 21 lines

```
typedef double d; // for N ~ 1e7; long double for N ~ 1
    e9
pair<ll, ll> approximate(d x, ll N) {
  ll LP = 0, LQ = 1, P = 1, Q = 0, inf = LLONG_MAX; d y
      = x;
  for (;;) {
    ll lim = min(P ? (N-LP) / P : inf, Q ? (N-LQ) / Q :
        inf),
      a = (ll)floor(y), b = min(a, lim),
      NP = b*P + LP, NQ = b*Q + LQ;
    if (a > b) {
      // If b > a/2, we have a semi-convergent that
          gives us a
      // better approximation; if b = a/2, we *may* have
          one.
      // Return {P, Q} here for a more canonical
          approximation.
      return (abs(x - (d)NP / (d)NQ) < abs(x - (d)P / (d
          )Q)) ?
        make_pair(NP, NQ) : make_pair(P, Q);
    }
    if (abs(y = 1/(y - (d)a)) > 3*N) {
      return {NP, NQ};
    }
    LP = P; P = NP;
    LQ = Q; Q = NQ;
  }
}
```

FracBinarySearch.h

Description: Given f and N, finds the smallest fraction $p/q \in [0, 1]$ such that $f(p/q)$ is true, and $p, q \le N$. You may want to throw an exception from f if it finds an exact solution, in which case N can be removed.
Usage: fracBS([](Frac f) { return f.p>=3*f.q; }, 10); // {1,3}
Time: $\mathcal{O}(\log(N))$
Status: stress-tested for n <= 300

27ab3e, 24 lines

```
struct Frac { ll p, q; };
template<class F>
Frac fracBS(F f, ll N) {
  bool dir = 1, A = 1, B = 1;
  Frac lo{0, 1}, hi{1, 1}; // Set hi to 1/0 to search
      (0, N]
  if (f(lo)) return lo;
  assert(f(hi));
  while (A || B) {
```

```
    ll adv = 0, step = 1; // move hi if dir, else lo
    for (int si = 0; step; (step *= 2) >>= si) {
      adv += step;
      Frac mid{lo.p * adv + hi.p, lo.q * adv + hi.q};
      if (abs(mid.p) > N || mid.q > N || dir == !f(mid))
          {
        adv -= step; si = 2;
      }
    }
    hi.p += lo.p * adv;
    hi.q += lo.q * adv;
    dir = !dir;
    swap(lo, hi);
    A = B; B = !!adv;
  }
  return dir ? hi : lo;
}
```

4.5 Pythagorean Triples

The Pythagorean triples are uniquely generated by

$$a = k \cdot (m^2 - n^2),\ \ b = k \cdot (2mn),\ \ c = k \cdot (m^2 + n^2),$$

with $m > n > 0$, $k > 0$, $m \perp n$, and either m or n even.

4.6 Primes

$p = 962592769$ is such that $2^{21} \mid p - 1$, which may be useful. For hashing use 970592641 (31-bit number), 31443539979727 (45-bit), 3006703054056749 (52-bit). There are 78498 primes less than 1 000 000.

Primitive roots exist modulo any prime power p^a, except for $p = 2, a > 2$, and there are $\phi(\phi(p^a))$ many. For $p = 2, a > 2$, the group $\mathbb{Z}_{2^a}^\times$ is instead isomorphic to $\mathbb{Z}_2 \times \mathbb{Z}_{2^{a-2}}$.

4.7 Estimates

$\sum_{d|n} d = O(n \log \log n)$.

The number of divisors of n is at most around 100 for $n < 5e4$, 500 for $n < 1e7$, 2000 for $n < 1e10$, 200 000 for $n < 1e19$.

4.8 Mobius Function

$$\mu(n) = \begin{cases} 0 & n \text{ is not square free} \\ 1 & n \text{ has even number of prime factors} \\ -1 & n \text{ has odd number of prime factors} \end{cases}$$

Mobius Inversion:

$$g(n) = \sum_{d|n} f(d) \Leftrightarrow f(n) = \sum_{d|n} \mu(d) g(n/d)$$

Other useful formulas/forms:

$\sum_{d|n} \mu(d) = [n = 1]$ (very useful)

$g(n) = \sum_{n|d} f(d) \Leftrightarrow f(n) = \sum_{n|d} \mu(d/n) g(d)$

$g(n) = \sum_{1 \le m \le n} f(\lfloor \frac{n}{m} \rfloor) \Leftrightarrow f(n) = \sum_{1 \le m \le n} \mu(m) g(\lfloor \frac{n}{m} \rfloor)$

Eratosthenes.h

Description: Prime sieve for generating all primes up to a certain limit. isprime[i] is true iff i is a prime.

Time: lim=100'000'000 $\approx$ 0.8 s. Runs 30% faster if only odd indices are stored.

Status: Tested

7c144c, 11 lines

```
const int MAX_PR = 5'000'000;
bitset<MAX_PR> isprime;
vi eratosthenesSieve(int lim) {
  isprime.set(); isprime[0] = isprime[1] = 0;
  for (int i = 4; i < lim; i += 2) isprime[i] = 0;
  for (int i = 3; i*i < lim; i += 2) if (isprime[i])
    for (int j = i*i; j < lim; j += i*2) isprime[j] = 0;
  vi pr;
  rep(i,2,lim) if (isprime[i]) pr.push_back(i);
  return pr;
}
```

Numerical (5)

5.1 Polynomials and recurrences

Polynomial.h

c9b7b0, 17 lines

```
struct Poly {
  vector<double> a;
  double operator()(double x) const {
    double val = 0;
    for (int i = sz(a); i--;) (val *= x) += a[i];
    return val;
  }
  void diff() {
    rep(i,1,sz(a)) a[i-1] = i*a[i];
    a.pop_back();
  }
  void divroot(double x0) {
    double b = a.back(), c; a.back() = 0;
    for(int i=sz(a)-1; i--;) c = a[i], a[i] = a[i+1]*x0+
        b, b=c;
    a.pop_back();
  }
};
```

PolyRoots.h

Description: Finds the real roots to a polynomial.

Usage: `polyRoots({{2,-3,1}},-1e9,1e9) // solve x^2-3x+2 = 0`

Time: $\mathcal{O}\left(n^2 \log(1/\epsilon)\right)$

"Polynomial.h" b00bfe, 23 lines

```
vector<double> polyRoots(Poly p, double xmin, double
    xmax) {
  if (sz(p.a) == 2) { return {-p.a[0]/p.a[1]}; }
  vector<double> ret;
  Poly der = p;
  der.diff();
  auto dr = polyRoots(der, xmin, xmax);
  dr.push_back(xmin-1);
  dr.push_back(xmax+1);
  sort(all(dr));
  rep(i,0,sz(dr)-1) {
    double l = dr[i], h = dr[i+1];
    bool sign = p(l) > 0;
    if (sign ^ (p(h) > 0)) {
      rep(it,0,60) { // while (h - l > 1e-8)
        double m = (l + h) / 2, f = p(m);
        if ((f <= 0) ^ sign) l = m;
        else h = m;
      }
      ret.push_back((l + h) / 2);
    }
  }
  return ret;
}
```

PolyInterpolate.h

Description: Given n points (x[i], y[i]), computes an n-1-degree polynomial p that passes through them: $p(x) = a[0] * x^0 + ... + a[n-1] * x^{n-1}$. For numerical precision, pick $x[k] = c * \cos(k/(n-1) * \pi), k = 0 \ldots n-1$.

Time: $\mathcal{O}\left(n^2\right)$

08bf48, 13 lines

```
typedef vector<double> vd;
vd interpolate(vd x, vd y, int n) {
  vd res(n), temp(n);
  rep(k,0,n-1) rep(i,k+1,n)
    y[i] = (y[i] - y[k]) / (x[i] - x[k]);
  double last = 0; temp[0] = 1;
  rep(k,0,n) rep(i,0,n) {
    res[i] += y[k] * temp[i];
    swap(last, temp[i]);
    temp[i] -= last * x[k];
  }
  return res;
}
```

BerlekampMassey.h

Description: Recovers any n-order linear recurrence relation from the first $2n$ terms of the recurrence. Useful for guessing linear recurrences after brute-forcing the first terms. Should work on any field, but numerical stability for floats is not guaranteed. Output will have size $\le n$.

Usage: `berlekampMassey({0, 1, 1, 3, 5, 11}) // {1, 2}`

Time: $\mathcal{O}\left(N^2\right)$

Status: bruteforce-tested mod 5 for n <= 5 and all s

"../number-theory/ModPow.h" 96548b, 18 lines

```
vector<ll> berlekampMassey(vector<ll> s) {
  int n = sz(s), L = 0, m = 0;
  vector<ll> C(n), B(n), T;
  C[0] = B[0] = 1;
  ll b = 1;
  rep(i,0,n) { ++m;
    ll d = s[i] % mod;
    rep(j,1,L+1) d = (d + C[j] * s[i - j]) % mod;
    if (!d) continue;
    T = C; ll coef = d * modpow(b, mod-2) % mod;
    rep(j,m,n) C[j] = (C[j] - coef * B[j - m]) % mod;
    if (2 * L > i) continue;
    L = i + 1 - L; B = T; b = d; m = 0;
  }
  C.resize(L + 1); C.erase(C.begin());
  for (ll& x : C) x = (mod - x) % mod;
  return C;
}
```

LinearRecurrence.h

Description: Generates the k'th term of an n-order linear recurrence $S[i] = \sum_j S[i-j-1]tr[j]$, given $S[0 \dots \geq n-1]$ and $tr[0 \dots n-1]$. Faster than matrix multiplication. Useful together with Berlekamp–Massey.

Usage: `linearRec({0, 1}, {1, 1}, k) // k'th Fibonacci number`

Time: $\mathcal{O}\left(n^2 \log k\right)$

Status: bruteforce-tested mod 5 for n <= 5

f4e444, 22 lines

```
typedef vector<ll> Poly;
ll linearRec(Poly S, Poly tr, ll k) {
  int n = sz(tr);
  auto combine = [&](Poly a, Poly b) {
    Poly res(n * 2 + 1);
    rep(i,0,n+1) rep(j,0,n+1)
      res[i + j] = (res[i + j] + a[i] * b[j]) % mod;
    for (int i = 2 * n; i > n; --i) rep(j,0,n)
      res[i - 1 - j] = (res[i - 1 - j] + res[i] * tr[j])
          % mod;
    res.resize(n + 1);
    return res;
  };
  Poly pol(n + 1), e(pol);
  pol[0] = e[1] = 1;
  for (++k; k; k /= 2) {
    if (k % 2) pol = combine(pol, e);
    e = combine(e, e);
  }
  ll res = 0;
  rep(i,0,n) res = (res + pol[i + 1] * S[i]) % mod;
  return res;
}
```

5.2 Optimization

GoldenSectionSearch.h

Description: Finds the argument minimizing the function f in the interval $[a, b]$ assuming f is unimodal on the interval, i.e. has only one local minimum and no local maximum. The maximum error in the result is *eps*. Works equally well for maximization with a small change in the code. See TernarySearch.h in the Various chapter for a discrete version.

Usage: `double func(double x) { return 4+x+.3*x*x; }`
`double xmin = gss(-1000,1000,func);`

Time: $\mathcal{O}\left(\log((b-a)/\epsilon)\right)$

Status: tested

31d45b, 14 lines

```
double gss(double a, double b, double (*f)(double)) {
  double r = (sqrt(5)-1)/2, eps = 1e-7;
  double x1 = b - r*(b-a), x2 = a + r*(b-a);
  double f1 = f(x1), f2 = f(x2);
  while (b-a > eps)
    if (f1 < f2) { //change to > to find maximum
      b = x2; x2 = x1; f2 = f1;
      x1 = b - r*(b-a); f1 = f(x1);
    } else {
      a = x1; x1 = x2; f1 = f2;
      x2 = a + r*(b-a); f2 = f(x2);
    }
  return a;
}
```

Simplex.h

Description: Solves a general linear maximization problem: maximize $c^T x$ subject to $Ax \leq b$, $x \geq 0$. Returns -inf if there is no solution, inf if there are arbitrarily good solutions, or the maximum value of $c^T x$ otherwise. The input vector is set to an optimal x (or in the unbounded case, an arbitrary solution fulfilling the constraints). Numerical stability is not guaranteed. For better performance, define variables such that $x = 0$ is viable.

Usage: `vvd A = {{1,-1}, {-1,1}, {-1,-2}};`
`vd b = {1,1,-4}, c = {-1,-1}, x;`
`T val = LPSolver(A, b, c).solve(x);`

Time: $\mathcal{O}\left(NM * \#pivots\right)$, where a pivot may be e.g. an edge relaxation. $\mathcal{O}\left(2^n\right)$ in the general case.

Status: seems to work?

aa8530, 62 lines

```
typedef double T; // long double, Rational, double + mod
    <P>...
typedef vector<T> vd;
typedef vector<vd> vvd;
const T eps = 1e-8, inf = 1/.0;
#define MP make_pair
#define ltj(X) if(s == -1 || MP(X[j],N[j]) < MP(X[s],N[s
    ])) s=j
struct LPSolver {
  int m, n;
  vi N, B;
  vvd D;
  LPSolver(const vvd& A, const vd& b, const vd& c) :
    m(sz(b)), n(sz(c)), N(n+1), B(m), D(m+2, vd(n+2)) {
      rep(i,0,m) rep(j,0,n) D[i][j] = A[i][j];
      rep(i,0,m) { B[i] = n+i; D[i][n] = -1; D[i][n+1] =
          b[i];}
      rep(j,0,n) { N[j] = j; D[m][j] = -c[j]; }
      N[n] = -1; D[m+1][n] = 1;
    }
  void pivot(int r, int s) {
    T *a = D[r].data(), inv = 1 / a[s];
    rep(i,0,m+2) if (i != r && abs(D[i][s]) > eps) {
      T *b = D[i].data(), inv2 = b[s] * inv;
      rep(j,0,n+2) b[j] -= a[j] * inv2;
      b[s] = a[s] * inv2;
    }
    rep(j,0,n+2) if (j != s) D[r][j] *= inv;
    rep(i,0,m+2) if (i != r) D[i][s] *= -inv;
    D[r][s] = inv;
    swap(B[r], N[s]);
  }
  bool simplex(int phase) {
    int x = m + phase - 1;
    for (;;) {
      int s = -1;
      rep(j,0,n+1) if (N[j] != -phase) ltj(D[x]);
      if (D[x][s] >= -eps) return true;
      int r = -1;
      rep(i,0,m) {
        if (D[i][s] <= eps) continue;
        if (r == -1 || MP(D[i][n+1] / D[i][s], B[i])
              < MP(D[r][n+1] / D[r][s], B[r])) r = i;
      }
      if (r == -1) return false;
      pivot(r, s);
    }
  }
  T solve(vd &x) {
    int r = 0;
    rep(i,1,m) if (D[i][n+1] < D[r][n+1]) r = i;
    if (D[r][n+1] < -eps) {
      pivot(r, n);
      if (!simplex(2) || D[m+1][n+1] < -eps) return -inf
          ;
      rep(i,0,m) if (B[i] == -1) {
        int s = 0;
        rep(j,1,n+1) ltj(D[i]);
        pivot(i, s);
      }
    }
    bool ok = simplex(1); x = vd(n);
    rep(i,0,m) if (B[i] < n) x[B[i]] = D[i][n+1];
    return ok ? D[m][n+1] : inf;
  }
};
```

5.3 Matrices

Determinant.h

Description: Calculates determinant of a matrix. Destroys the matrix.

Time: $\mathcal{O}\left(N^3\right)$

Status: somewhat tested

bd5cec, 15 lines

```
double det(vector<vector<double>>& a) {
  int n = sz(a); double res = 1;
  rep(i,0,n) {
    int b = i;
    rep(j,i+1,n) if (fabs(a[j][i]) > fabs(a[b][i])) b =
        j;
    if (i != b) swap(a[i], a[b]), res *= -1;
    res *= a[i][i];
    if (res == 0) return 0;
    rep(j,i+1,n) {
      double v = a[j][i] / a[i][i];
      if (v != 0) rep(k,i+1,n) a[j][k] -= v * a[i][k];
    }
  }
  return res;
}
```

IntDeterminant.h

Description: Calculates determinant using modular arithmetics. Modulos can also be removed to get a pure-integer version.
Time: $\mathcal{O}\left(N^3\right)$
Status: bruteforce-tested for N <= 3, mod <= 7

3313dc, 18 lines

```
const ll mod = 12345;
ll det(vector<vector<ll>>& a) {
  int n = sz(a); ll ans = 1;
  rep(i,0,n) {
    rep(j,i+1,n) {
      while (a[j][i] != 0) { // gcd step
        ll t = a[i][i] / a[j][i];
        if (t) rep(k,i,n)
          a[i][k] = (a[i][k] - a[j][k] * t) % mod;
        swap(a[i], a[j]);
        ans *= -1;
      }
    }
    ans = ans * a[i][i] % mod;
    if (!ans) return 0;
  }
  return (ans + mod) % mod;
}
```

GaussElimination.h

Description: Gauss elimation to solve system of linear equations
Usage: variables are numbered 0 -> m-1, the m-th column is the right hand side
Time: $\mathcal{O}\left(min(N,M) \cdot N \cdot M\right)$ where N = equations, M = variables
Status: Tested on VNOI

9990b6, 31 lines

```
int gauss(vector<vector<double>> &a, vector<double> &ans
    ) {
  const double EPS = 1e-9;
  int n = a.size(), m = a[0].size() - 1;
  vector<int> where(m, -1);
  for (int col = 0, row = 0; col < m && row < n; ++col)
      {
    int p = row;
    for (int i = row; i < n; ++i)
      if (abs(a[i][col]) > abs(a[p][col])) p = i;
    if (abs(a[p][col]) < EPS) continue;
    swap(a[row], a[p]);
    where[col] = row;
    for (int i = 0; i < n; ++i) if (i != row) {
      double c = a[i][col] / a[row][col];
      for (int j = col; j <= m; ++j)
        a[i][j] -= a[row][j] * c;
    }
    ++row;
  }
  ans.assign(m, 0);
  for (int i = 0; i < m; ++i) if (where[i] != -1) {
    ans[i] = a[where[i]][m] / a[where[i]][i];
  }
  for (int i = 0; i < n; ++i) {
    double sum = 0;
    for (int j = 0; j < m; ++j) sum += ans[j] * a[i][j];
    if (abs(sum - a[i][m]) > EPS) return 0;
  }
  for (int i = 0; i < m; ++i)
    if (where[i] == -1) return 2;
  return 1;
}
```

GaussElimination2.h

Description: Gauss elimation to solve system of linear equations (mod 2)
Usage: all roots of the system have the form: ans + sum(c(i) * basis(i))
Time: $\mathcal{O}\left(\frac{min(N,M) \cdot N \cdot M}{64}\right)$ where N = equations, M = variables
Status: Tested on judge.yosupo.jp

1a50a3, 49 lines

```
namespace Gauss {
  // n: num equations, m: num variables
  // left hand side: [0, m-1], right hand side: column m
  const int M = 4096 + 5;
  using T = bitset<M>;
  int n, m;
  int where[M];
  vector<T> a;
  T ans;
  void init(int _n, int _m, const vector<T> &_a) {
    n = _n, m = _m, a = _a;
  }
  int gauss() {
    ans.reset();
    for (int i = 0; i < m; ++i) where[i] = -1;
    int row = 0;
    for (int col = 0; col < m && row < n; ++col) {
      for (int i = row; i < n; ++i) if (a[i][col]) {
        swap(a[i], a[row]);
        break;
      }
      if (!a[row][col]) continue;
      where[col] = row;
      for (int i = 0; i < n; ++i)
        if (i != row && a[i][col]) a[i] ^= a[row];
      ++row;
    }
    for (int i = 0; i < m; ++i) if (where[i] != -1) {
      ans[i] = a[where[i]][m];
    }
    for (int i = 0; i < n; ++i) {
      bool sum = 0;
      for (int j = 0; j < m; ++j) sum ^= ans[j] & a[i][j
          ];
      if (sum != a[i][m]) return -1;
    }
    return row;
  }
  vector<T> get_basis() { // assume gauss() has been
      called
    vector<T> basis;
    for (int i = 0; i < m; ++i) if (where[i] == -1) {
      T x; x[i] = 1;
      // x(j) + a[where[j]][i] * x[i] = 0
      for (int j = 0; j < m; ++j)
        if (where[j] != -1) x[j] = a[where[j]][i];
      basis.emplace_back(x);
    }
    return basis;
  }
}
```

MatrixInverse.h

Description: Invert matrix A. Returns rank; result is stored in A unless singular (rank < n). Can easily be extended to prime moduli; for prime powers, repeatedly set $A^{-1} = A^{-1}(2I - AA^{-1})$ (mod p^k) where A^{-1} starts as the inverse of A mod p, and k is doubled in each step.
Time: $\mathcal{O}\left(n^3\right)$
Status: Slightly tested

ebfff6, 32 lines

```
int matInv(vector<vector<double>>& A) {
  int n = sz(A); vi col(n);
  vector<vector<double>> tmp(n, vector<double>(n));
  rep(i,0,n) tmp[i][i] = 1, col[i] = i;
  rep(i,0,n) {
    int r = i, c = i;
    rep(j,i,n) rep(k,i,n)
      if (fabs(A[j][k]) > fabs(A[r][c]))
        r = j, c = k;
    if (fabs(A[r][c]) < 1e-12) return i;
    A[i].swap(A[r]); tmp[i].swap(tmp[r]);
    rep(j,0,n)
      swap(A[j][i], A[j][c]), swap(tmp[j][i], tmp[j][c])
          ;
    swap(col[i], col[c]);
    double v = A[i][i];
```

```
    rep(j,i+1,n) {
      double f = A[j][i] / v;
      A[j][i] = 0;
      rep(k,i+1,n) A[j][k] -= f*A[i][k];
      rep(k,0,n) tmp[j][k] -= f*tmp[i][k];
    }
    rep(j,i+1,n) A[i][j] /= v;
    rep(j,0,n) tmp[i][j] /= v;
    A[i][i] = 1;
  }
  for (int i = n-1; i > 0; --i) rep(j,0,i) {
    double v = A[j][i];
    rep(k,0,n) tmp[j][k] -= v*tmp[i][k];
  }
  rep(i,0,n) rep(j,0,n) A[col[i]][col[j]] = tmp[i][j];
  return n;
}
```

MatrixInverse-mod.h

Description: Invert matrix A modulo a prime. Returns rank; result is stored in A unless singular (rank < n). For prime powers, repeatedly set $A^{-1} = A^{-1}(2I - AA^{-1}) \pmod{p^k}$ where A^{-1} starts as the inverse of A mod p, and k is doubled in each step.
Time: $\mathcal{O}(n^3)$
Status: Slightly tested

"../number-theory/ModPow.h" 0b7b13, 34 lines

```
int matInv(vector<vector<ll>>& A) {
  int n = sz(A); vi col(n);
  vector<vector<ll>> tmp(n, vector<ll>(n));
  rep(i,0,n) tmp[i][i] = 1, col[i] = i;
  rep(i,0,n) {
    int r = i, c = i;
    rep(j,i,n) rep(k,i,n) if (A[j][k]) {
      r = j; c = k; goto found;
    }
    return i;
found:
    A[i].swap(A[r]); tmp[i].swap(tmp[r]);
    rep(j,0,n)
      swap(A[j][i], A[j][c]), swap(tmp[j][i], tmp[j][c])
          ;
    swap(col[i], col[c]);
    ll v = modpow(A[i][i], mod - 2);
    rep(j,i+1,n) {
      ll f = A[j][i] * v % mod;
      A[j][i] = 0;
      rep(k,i+1,n) A[j][k] = (A[j][k] - f*A[i][k]) % mod
          ;
      rep(k,0,n) tmp[j][k] = (tmp[j][k] - f*tmp[i][k]) %
           mod;
    }
    rep(j,i+1,n) A[i][j] = A[i][j] * v % mod;
    rep(j,0,n) tmp[i][j] = tmp[i][j] * v % mod;
    A[i][i] = 1;
  }
  for (int i = n-1; i > 0; --i) rep(j,0,i) {
    ll v = A[j][i];
    rep(k,0,n) tmp[j][k] = (tmp[j][k] - v*tmp[i][k]) %
        mod;
  }
  rep(i,0,n) rep(j,0,n)
    A[col[i]][col[j]] = tmp[i][j] % mod + (tmp[i][j] <
        0)*mod;
  return n;
}
```

Tridiagonal.h

Description: $x = \text{tridiagonal}(d, p, q, b)$ solves the equation system

$$\begin{pmatrix} b_0 \\ b_1 \\ b_2 \\ b_3 \\ \vdots \\ b_{n-1} \end{pmatrix} = \begin{pmatrix} d_0 & p_0 & 0 & 0 & \cdots & 0 \\ q_0 & d_1 & p_1 & 0 & \cdots & 0 \\ 0 & q_1 & d_2 & p_2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \ddots & \ddots & \vdots \\ 0 & 0 & \cdots & q_{n-3} & d_{n-2} & p_{n-2} \\ 0 & 0 & \cdots & 0 & q_{n-2} & d_{n-1} \end{pmatrix} \begin{pmatrix} x_0 \\ x_1 \\ x_2 \\ x_3 \\ \vdots \\ x_{n-1} \end{pmatrix}.$$

This is useful for solving problems on the type

$$a_i = b_i a_{i-1} + c_i a_{i+1} + d_i,\ 1 \le i \le n,$$

where a_0, a_{n+1}, b_i, c_i and d_i are known. a can then be obtained from

$$\{a_i\} = \text{tridiagonal}(\{1,-1,-1,...,-1,1\}, \{0,c_1,c_2,\ldots,c_n\}, \{b_1,b_2,\ldots,b_n,0\}, \{a_0,d_1,d_2,\ldots,d_n,a_{n+1}\}).$$

Fails if the solution is not unique. If $|d_i| > |p_i| + |q_{i-1}|$ for all i, or $|d_i| > |p_{i-1}| + |q_i|$, or the matrix is positive definite, the algorithm is numerically stable and neither `tr` nor the check for `diag[i] == 0` is needed.
Time: $\mathcal{O}(N)$
Status: Brute-force tested mod 5 and 7 and stress-tested for real matrices obeying the criteria above.

8f9fa8, 26 lines

```
typedef double T;
vector<T> tridiagonal(vector<T> diag, const vector<T>&
    super,
    const vector<T>& sub, vector<T> b) {
  int n = sz(b); vi tr(n);
  rep(i,0,n-1) {
    if (abs(diag[i]) < 1e-9 * abs(super[i])) { // diag[i
        ] == 0
      b[i+1] -= b[i] * diag[i+1] / super[i];
      if (i+2 < n) b[i+2] -= b[i] * sub[i+1] / super[i];
      diag[i+1] = sub[i]; tr[++i] = 1;
    } else {
      diag[i+1] -= super[i]*sub[i]/diag[i];
      b[i+1] -= b[i]*sub[i]/diag[i];
    }
  }
  for (int i = n; i--;) {
    if (tr[i]) {
      swap(b[i], b[i-1]);
      diag[i-1] = diag[i];
      b[i] /= super[i-1];
    } else {
      b[i] /= diag[i];
      if (i) b[i-1] -= b[i]*super[i-1];
    }
  }
  return b;
}
```

5.4 Fourier transforms

FastFourierTransform.h

Description: fft(a) computes $\hat{f}(k) = \sum_x a[x] \exp(2\pi i \cdot kx/N)$ for all k. N must be a power of 2. Useful for convolution: `conv(a, b) = c`, where $c[x] = \sum a[i]b[x-i]$. For convolution of complex numbers or more than two vectors: FFT, multiply pointwise, divide by n, reverse(start+1, end), FFT back. Rounding is safe if $(\sum a_i^2 + \sum b_i^2)\log_2 N < 9 \cdot 10^{14}$ (in practice 10^{16}; higher for random inputs). Otherwise, use NTT/FFTMod.
Time: $\mathcal{O}(N \log N)$ with $N = |A| + |B|$ ($\sim 1s$ for $N = 2^{22}$)
Status: somewhat tested

00ced6, 35 lines

```
typedef complex<double> C;
typedef vector<double> vd;
void fft(vector<C>& a) {
  int n = sz(a), L = 31 - __builtin_clz(n);
  static vector<complex<long double>> R(2, 1);
  static vector<C> rt(2, 1);  // (^ 10% faster if double
      )
  for (static int k = 2; k < n; k *= 2) {
    R.resize(n); rt.resize(n);
    auto x = polar(1.0L, acos(-1.0L) / k);
    rep(i,k,2*k) rt[i] = R[i] = i&1 ? R[i/2] * x : R[i
        /2];
  }
  vi rev(n);
  rep(i,0,n) rev[i] = (rev[i / 2] | (i & 1) << L) / 2;
  rep(i,0,n) if (i < rev[i]) swap(a[i], a[rev[i]]);
  for (int k = 1; k < n; k *= 2)
    for (int i = 0; i < n; i += 2 * k) rep(j,0,k) {
      C z = rt[j+k] * a[i+j+k]; // (25% faster if hand-
          rolled)
      a[i + j + k] = a[i + j] - z;
      a[i + j] += z;
    }
}
vd conv(const vd& a, const vd& b) {
  if (a.empty() || b.empty()) return {};
  vd res(sz(a) + sz(b) - 1);
  int L = 32 - __builtin_clz(sz(res)), n = 1 << L;
  vector<C> in(n), out(n);
  copy(all(a), begin(in));
  rep(i,0,sz(b)) in[i].imag(b[i]);
  fft(in);
  for (C& x : in) x *= x;
  rep(i,0,n) out[i] = in[-i & (n - 1)] - conj(in[i]);
  fft(out);
  rep(i,0,sz(res)) res[i] = imag(out[i]) / (4 * n);
  return res;
```

```
}
```

FastFourierTransformMod.h

Description: FFT to multiply two polynomials.
Usage: `Set MOD. Rounded convolution coefficients must fit in a signed 64-bit integer.`
Time: $\mathcal{O}(N \log N)$ where N is the total sizes of two polynomials
Status: Well-tested

4551f1, 69 lines

```
namespace FFT {
  using ld = long double;
  const int MOD = 1000000007; // Set to the required
      modulus.
  const ld pi = atan2(1, 1) * 4;
  struct cmplx {
    ld a, b;
    cmplx() {
      a = 0, b = 0;
    }
    cmplx(ld a1, ld b1) {
      a = a1, b = b1;
    }
  };
  inline cmplx sum(cmplx& c1, cmplx& c2) {
    return {c1.a + c2.a, c1.b + c2.b};
  }
  inline cmplx sub(cmplx& c1, cmplx& c2) {
    return {c1.a - c2.a, c1.b - c2.b};
  }
  inline cmplx mult(cmplx& c1, cmplx& c2) {
    return {c1.a * c2.a - c1.b * c2.b, c1.a * c2.b + c1.
        b * c2.a};
  }
  void fft(vector<cmplx> &a, int n) {
    for (int i = 1, j = 0; i < n; ++i) {
      int bit = n >> 1;
      for (; j >= bit; bit >>= 1) j -= bit;
      j += bit;
      if (i < j) swap(a[i], a[j]);
    }
    for (int k = 1; k < n; k *= 2) {
      ld phi = pi / k;
      cmplx wt(cos(phi), sin(phi));
      for (int i = 0; i < n; i += 2 * k) {
        cmplx w(1, 0);
        for (int j = 0; j < k; ++j) {
          cmplx u = a[i + j], v = mult(a[i + j + k], w);
          a[i + j] = sum(u, v);
          a[i + j + k] = sub(u, v);
          w = mult(w, wt);
        }
      }
    }
  }
  vector<int> multiply(vector<int> &s, vector<int> &t) {
    if (s.empty() || t.empty()) return {};
    int need = (int) s.size() + t.size() - 1, sz = 1;
    while (sz < need) sz <<= 1;
    vector<cmplx> a(sz), b(sz);
    for (int i = 0; i < sz; ++i) {
      if(i < (int)s.size()) a[i] = cmplx(s[i], 0);
      else a[i] = cmplx(0, 0);
      if(i < (int)t.size()) b[i] = cmplx(t[i], 0);
      else b[i] = cmplx(0, 0);
    }
    fft(a, sz);
    fft(b, sz);
    for (int i = 0; i < sz; ++i) {
      a[i] = mult(a[i], b[i]);
    }
    fft(a, sz);
    reverse(a.begin() + 1, a.end());
    vector<int> ans(need);
    for (int i = 0; i < need; ++i) ans[i] = (llround(a[i
        ].a / sz) % MOD + MOD) % MOD;
    return ans;
  }
}
vector<int> operator * (vector<int> &s, vector<int> &t)
    {
  return FFT::multiply(s, t);
}
```

NumberTheoreticTransform.h

Description: ntt(a) computes $\hat{f}(k) = \sum_x a[x] g^{xk}$ for all k, where $g = \text{root}^{(mod-1)/N}$. N must be a power of 2. Useful for convolution modulo specific nice primes of the form $2^a b + 1$, where the convolution result has size at most 2^a. For arbitrary modulo, see FFTMod. `conv(a, b) = c`, where $c[x] = \sum a[i]b[x-i]$. For manual convolution: NTT the inputs, multiply pointwise, divide by n, reverse(start+1, end), NTT back. Inputs must be in [0, mod).
Time: $\mathcal{O}(N \log N)$
Status: stress-tested

ccea62, 41 lines

```
const ll mod = (119 << 23) + 1, root = 62; // =
    998244353
ll nttPow(ll a, ll e) {
  ll res = 1;
  for (; e; e >>= 1, a = a * a % mod)
    if (e & 1) res = res * a % mod;
  return res;
}
// For p < 2^30 there is also e.g. 5 << 25, 7 << 26, 479
    << 21
// and 483 << 21 (same root). The last two are > 10^9.
typedef vector<ll> vl;
void ntt(vl &a) {
  int n = sz(a), L = 31 - __builtin_clz(n);
  static vl rt(2, 1);
  for (static int k = 2, s = 2; k < n; k *= 2, s++) {
    rt.resize(n);
    ll z[] = {1, nttPow(root, mod >> s)};
    rep(i,k,2*k) rt[i] = rt[i / 2] * z[i & 1] % mod;
  }
  vi rev(n);
  rep(i,0,n) rev[i] = (rev[i / 2] | (i & 1) << L) / 2;
  rep(i,0,n) if (i < rev[i]) swap(a[i], a[rev[i]]);
  for (int k = 1; k < n; k *= 2)
    for (int i = 0; i < n; i += 2 * k) rep(j,0,k) {
      ll z = rt[j + k] * a[i + j + k] % mod, &ai = a[i +
          j];
      a[i + j + k] = ai - z + (z > ai ? mod : 0);
      ai += (ai + z >= mod ? z - mod : z);
    }
}
vl conv(const vl &a, const vl &b) {
  if (a.empty() || b.empty()) return {};
  int s = sz(a) + sz(b) - 1, B = 32 - __builtin_clz(s),
    n = 1 << B;
  int inv = nttPow(n, mod - 2);
  vl L(a), R(b), out(n);
  L.resize(n), R.resize(n);
  ntt(L), ntt(R);
  rep(i,0,n)
    out[-i & (n - 1)] = (ll)L[i] * R[i] % mod * inv %
        mod;
  ntt(out);
  return {out.begin(), out.begin() + s};
}
```

NTTOnline.h

Description: Calculates NTT Online.
Time: $\mathcal{O}(N \log^2 N)$
Status: Probably Tested

8ac7e9, 73 lines

```
const int N = 1 << 18;
const int mod = 998244353;
const int root = 3;
int lim, rev[N], w[N], wn[N], inv_lim;
void reduce(int &x) { x = (x + mod) % mod; }
int POW(int x, int y, int ans = 1) {
  for (; y; y >>= 1, x = (long long) x * x % mod) if (y
      & 1) ans = (long long) ans * x % mod;
  return ans;
}
void precompute(int len) {
  lim = wn[0] = 1; int s = -1;
  while (lim < len) lim <<= 1, ++s;
  for (int i = 0; i < lim; ++i) rev[i] = rev[i >> 1] >>
      1 | (i & 1) << s;
  const int g = POW(root, (mod - 1) / lim);
  inv_lim = POW(lim, mod - 2);
  for (int i = 1; i < lim; ++i) wn[i] = (long long) wn[i
      - 1] * g % mod;
}
void ntt(vector<int> &a, int typ) {
  for (int i = 0; i < lim; ++i) if (i < rev[i]) swap(a[i
      ], a[rev[i]]);
  for (int i = 1; i < lim; i <<= 1) {
```

```
    for (int j = 0, t = lim / i / 2; j < i; ++j) w[j] =
        wn[j * t];
    for (int j = 0; j < lim; j += i << 1) {
      for (int k = 0; k < i; ++k) {
        const int x = a[k + j], y = (long long) a[k + j
            + i] * w[k] % mod;
        reduce(a[k + j] += y - mod), reduce(a[k + j + i]
            = x - y);
      }
    }
  }
  if (!typ) {
    reverse(a.begin() + 1, a.begin() + lim);
    for (int i = 0; i < lim; ++i) a[i] = (long long) a[i
        ] * inv_lim % mod;
  }
}
vector<int> multiply(vector<int> &f, vector<int> &g) {
  int n = (int)f.size() + (int)g.size() - 1;
  precompute(n);
  vector<int> a = f, b = g;
  a.resize(lim); b.resize(lim);
  ntt(a, 1), ntt(b, 1);
  for (int i = 0; i < lim; ++i) a[i] = (long long) a[i]
      * b[i] % mod;
  ntt(a, 0);
  return a;
}
// 1s for n = 2e5
struct OnlineConvolution {
  vector<int> a, b, c;
  int k;
  OnlineConvolution(int n): a(n), b(n), c(n), k(0) {}
  // poly c = poly a * poly b
  // add a[i] = x and b[i] = y and it will return c[i]
  // online!
  int extend(int i, int x, int y) {
    assert(i == k);
    a[k] = x;
    b[k] = y;
    int s = k + 2;
    for (int w = 1; s % w == 0 && w < s; w <<= 1) {
      for (int ri = 0; ri < 2; ri++) {
        if (ri == 0 || w * 2 != s) {
          vector<int> f(w), g(w);
          for (int i = 0; i < w; i++) f[i] = a[w - 1 + i
              ], g[i] = b[k - w + 1 + i];
          f = multiply(f, g);
          for (int i = 0, j = k; i < f.size() && j < c.
              size(); i++, j++) {
            c[j] += f[i];
            if (c[j] >= mod) c[j] -= mod;
          }
        }
        swap(a, b);
      }
    }
    return c[k++];
  }
};
```

FastSubsetTransform.h

Description: Transform to a basis with fast convolutions of the form $c[z] = \sum_{z=x \oplus y} a[x] \cdot b[y]$, where $\oplus$ is one of AND, OR, XOR. The size of a must be a power of two.
Time: $\mathcal{O}(N \log N)$
Status: stress-tested

464cf3, 16 lines

```
void FST(vi& a, bool inv) {
  for (int n = sz(a), step = 1; step < n; step *= 2) {
    for (int i = 0; i < n; i += 2 * step) rep(j,i,i+step
        ) {
      int &u = a[j], &v = a[j + step]; tie(u, v) =
        inv ? pii(v - u, u) : pii(v, u + v); // AND
        inv ? pii(v, u - v) : pii(u + v, u); // OR
        pii(u + v, u - v);                   // XOR
    }
  }
  if (inv) for (int& x : a) x /= sz(a); // XOR only
}
vi conv(vi a, vi b) {
  FST(a, 0); FST(b, 0);
  rep(i,0,sz(a)) a[i] *= b[i];
  FST(a, 1); return a;
}
```

Data structures (6)

OrderSet.h

Description: Ordered set.
Usage: 0-indexed.
find_by_order(x) returns x-th element (ascending order)
order_of_key(x) returns no. of elements <x.
Time: $\mathcal{O}(\log N)$ for each operation
Status: Tested on one problem.

<ext/pb_ds/assoc_container.hpp> df383f, 3 lines

```
using namespace __gnu_pbds;
template <class T>
using orderset = tree<T, null_type, less<T>, rb_tree_tag
    , tree_order_statistics_node_update>;
```

FastSet.h

Description: Fast set for sets that only have integers in [1, N]
Usage: call init() first
Memory: $\mathcal{O}(N)$
Time: $\mathcal{O}(log64(n))$ for update and lowerbound, upperbound
Status: Well-tested

e52539, 85 lines

```
struct FastSet {
  static const int N = 250000 + 5;
  static const int BLOCK = 64;
  static const int LAYER = 3;
  uint64_t a[N + 5]{};
  int layer_start[LAYER];
  FastSet(){
    int start = 0, pos = 1;
    for(int i = 0; i < LAYER; ++i) {
      layer_start[i] = start;
      start += pos;
      pos = pos * BLOCK;
    }
  }
  void init() { fill(a, a + N + 5, 0); }
  void update(int x) {
    assert(1 <= x && x <= N);
    //add if x not in set and vice versa
    int id = x >> 6, prev = layer_start[LAYER - 1] + id;
    a[prev] ^= (1ULL << (x & (BLOCK - 1)));
    for (int i = LAYER - 2; i >= 0; --i) {
      x >>= 6, id >>= 6;
      int digit = x & (BLOCK - 1), cur = id +
          layer_start[i];
      if ((a[prev] > 0) != ((a[cur] >> digit) & 1ULL)) a
          [cur] ^= (1ULL << digit);
      prev = cur;
    }
  }
  int get_next(uint64_t mask, int l) {
    if (l >= BLOCK) return -1;
    mask &= ~uint64_t(0) << l;
    return mask ? __builtin_ctzll(mask) : -1;
  }
  int find_next(int x) { // return first ele >= x or -1
      if no ele
    int id = x >> 6;
    int foo = get_next(a[id + layer_start[LAYER - 1]], x
        & (BLOCK - 1));
    if (foo != -1) return x - (x & (BLOCK - 1)) + foo;
    x >>= 6, id >>= 6;
    for (int i = LAYER - 2; i>=0; --i) {
      int digit = (x & (BLOCK - 1)) + 1;
      int cur = layer_start[i] + id;
      int foo = get_next(a[cur], digit);
      if (foo != -1) {
        id = (id << 6) + foo;
        for (int j = i + 1; j < LAYER; ++j) {
          int digit = __builtin_ctzll(a[id + layer_start
              [j]]);
          id = (id << 6) + digit;
        }
        return id;
      }
      x >>= 6, id >>= 6;
    }
    return -1;
  }
  int get_prev(uint64_t mask, int l) {
    if (l < BLOCK) mask &= (1ULL << l) - 1;
```

```
    return mask ? 63 - __builtin_clzll(mask) : -1;
  }
  int find_prev(int x) { // return last ele < x or -1 if
       no ele
    int id = x >> 6;
    int foo = get_prev(a[id + layer_start[LAYER - 1]], x
         & (BLOCK - 1));
    if (foo != -1) return x - (x & (BLOCK - 1)) + foo;
    x >>= 6, id >>= 6;
    for(int i = LAYER - 2; i>=0; --i) {
      int digit = (x & (BLOCK - 1)) + 1;
      int cur = layer_start[i] + id;
      int foo = get_prev(a[cur], digit - 1);
      if (foo != -1) {
        id = (id << 6) + foo;
        for(int j = i + 1; j < LAYER; ++j) {
          int digit = (63 - __builtin_clzll(a[id +
              layer_start[j]]));
          id = (id << 6) + digit;
        }
        return id;
      }
      x >>= 6, id >>= 6;
    }
    return -1;
  }
  int get_min() {
    return find_next(1);
  }
  int get_max() {
    return find_prev(N + 1);
  }
};
```

LichaoTree.h

Description: Li Chao Tree, can query min and add line.
Time: Both operations are $\mathcal{O}(\log N)$
Status: Tested by KickingKun

764eb9, 43 lines

```
struct LiChaoTree {
  //range for x is in [-lim, lim].
  static const ll lim = 1e9, INF = 1e18;
  struct Line {
    ll a, b;
    ll operator() (ll x) {
      return a * x + b;
    }
  };
  struct node {
    Line line;
    node *l, *r;
    node() {
      line = {0, INF};
      l = r = NULL;
    }
  };
  node *root = new node();
  ll query(node *i, ll l, ll r, ll x) {
    if (i == NULL || x < l || x > r) return INF;
    ll m = (l + r) >> 1;
    ll ans = i->line(x);
    ans = min(ans, query(i->l, l, m, x));
    ans = min(ans, query(i->r, m + 1, r, x));
    return ans;
  }
  ll query(ll x) {
    return query(root, -lim, lim, x);
  }
  void add(Line li, node *&cur, ll l, ll r) {
    if (cur == nullptr){
      cur = new node(); cur->line = li;
      return;
    }
    ll mid = (l + r) >> 1;
    if (li(mid) < cur->line(mid)) swap(li, cur->line);
    if (li(l) < cur->line(l)) add(li, cur->l, l, mid);
    if (li(r) < cur->line(r)) add(li, cur->r, mid + 1, r
        );
  }
  void add(ll m, ll b) {
    add({m, b}, root, -lim, lim);
  }
} cht;
```

PersistentIT.h

Description: Persistent segment tree with point add updates and range sum queries.
Usage: `Remember to call mytree.resize(n) before any operations.`
`Make sure to check for integer overflows in update values.`
Memory: $\mathcal{O}(\log N \cdot numUpdates)$
Time: $\mathcal{O}(\log N)$ for each operation
Status: Tested on CSES

c22699, 46 lines

```
//change the parameters
const int MAX_UPDATES = 1e7+10;
struct Node {
  int le, ri;
  ll sum;
  Node(int _le = 0, int _ri = 0, ll _sum = 0) {
    le = _le, ri = _ri, sum = _sum;
  }
} nodes[MAX_UPDATES];
struct PersistentIT { // 1-indexed array
  int n;
  PersistentIT(int _n = 0) {
    resize(_n);
  }
  void resize(int _n) {
    n = _n;
  }
  int numNode = 0;
  int update(int oldId, int l, int r, int p, int val) {
    if (l > p || r < p) return oldId;
    assert(numNode + 1 < MAX_UPDATES);
    numNode++; int id = numNode;
    if (l == r) {
      nodes[id] = nodes[oldId];
      nodes[id].sum += val;
      return id;
    }
    int mid = (l + r) >> 1;
    nodes[id].le = update(nodes[oldId].le, l, mid, p,
        val);
    nodes[id].ri = update(nodes[oldId].ri, mid + 1, r, p
        , val);
    nodes[id].sum = nodes[nodes[id].le].sum + nodes[
        nodes[id].ri].sum;
    return id;
  }
  int update(int root, int p, int val) {
    return update(root, 1, n, p, val);
  }
  ll get(int id, int l, int r, int u, int v) {
    if (l > v || r < u) return 0;
    if (u <= l && r <= v) return nodes[id].sum;
    int mid = (l + r) >> 1;
    return get(nodes[id].le, l, mid, u, v) + get(nodes[
        id].ri, mid + 1, r, u, v);
  }
  ll get(int root, int u, int v) {
    return get(root, 1, n, u, v);
  }
} mytree;
```

PersistentLazyIT.h

Description: Persistent segment tree with range updates and range sum queries.
Usage: `Remember to call mytree.resize(n) before any operations.`
Memory: $\mathcal{O}(\log N \cdot numUpdates)$
Time: $\mathcal{O}(\log N)$ amortized per update, $\mathcal{O}(\log N)$ per query
Status: Randomized version-history regression test

e5c3a6, 65 lines

```
struct Node {
  int le, ri;
  ll lazy, sum;
  Node(int _le = 0, int _ri = 0, ll _lazy = 0, ll _sum =
       0) {
    le = _le, ri = _ri, lazy = _lazy, sum = _sum;
  }
};
vector<Node> nodes(1); // Index 0 is the all-zero
    subtree.
int numNode = 0;
vector<int> version;
struct PersistentLazyIT {
  int n;
  PersistentLazyIT(int _n = 0) {
    resize(_n);
```

```
}
void resize(int _n) {
  n = _n;
}
int newlazykid(int oldId, int l, int r, ll val) {
  int id = ++numNode;
  nodes.push_back(nodes[oldId]);
  nodes[id].sum += 1ll * val * (r - l + 1);
  nodes[id].lazy += val;
  return id;
}
int newparent(int le, int ri) {
  int id = ++numNode;
  nodes.emplace_back();
  nodes[id].le = le, nodes[id].ri = ri;
  nodes[id].sum = nodes[le].sum + nodes[ri].sum;
  return id;
}
void push(int id, int l, int r) {
  if (nodes[id].lazy == 0) return;
  if (l != r) {
    int mid = (l + r) >> 1;
    nodes[id].le = newlazykid(nodes[id].le, l, mid,
        nodes[id].lazy);
    nodes[id].ri = newlazykid(nodes[id].ri, mid + 1, r
        , nodes[id].lazy);
  }
  nodes[id].lazy = 0;
}
int update(int oldId, int l, int r, int u, int v, ll
    val) {
  if (l > v || r < u) return oldId;
  if (u <= l && r <= v) return newlazykid(oldId, l, r,
       val);
  int copy = newlazykid(oldId, l, r, 0);
  push(copy, l, r); int mid = (l + r) >> 1;
  int le = update(nodes[copy].le, l, mid, u, v, val),
    ri = update(nodes[copy].ri, mid + 1, r, u, v, val)
        ;
  return newparent(le, ri);
}
int update(int root, int u, int v, ll val) {
  assert(n > 0);
  return update(root, 1, n, u, v, val);
}
ll get(int id, int l, int r, int u, int v, ll carry =
    0) {
  if (l > v || r < u) return 0;
  if (u <= l && r <= v) return nodes[id].sum + carry *
       (r - l + 1);
  carry += nodes[id].lazy; int mid = (l + r) >> 1;
  return get(nodes[id].le, l, mid, u, v, carry) + get(
      nodes[id].ri, mid + 1, r, u, v, carry);
}
ll get(int root, int u, int v) {
  assert(n > 0);
    return get(root, 1, n, u, v);
  }
} mytree;
```

GPHashTable.h

Description: Hash map with mostly the same API as unordered_map, but ~3x faster. Uses 1.5x memory. Costs like 6MB for 1e5 ll, >500MB for 5e6 ll??? Initial capacity must be a power of 2 (if provided).

94c495, 7 lines

```
#include <bits/extc++.h>
// To use most bits rather than just the lowest ones:
struct chash { // large odd number for C
  const uint64_t C = ll(4e18 * acos(0)) | 71;
  ll operator()(ll x) const { return __builtin_bswap64(x
      *C); }
};
__gnu_pbds::gp_hash_table<ll,int,chash> ht({},{},{},{},{
    1<<16});
```

ConvexHullTrickMonotonicSlopes.h

Description: Fast convex hull trick for monotonic slopes
Time: $\mathcal{O}(1)$ for addition and $\mathcal{O}(1)$ or $\mathcal{O}(\log N)$ for query
Status: Well-tested

6042b6, 54 lines

```
// Adding decreasing slopes, query min
// Use get() if query increasing, query() otherwise
struct ConvexHullTrick {
  const ll INF = 1e18;
  struct Line {
    ll m, b;
    Line(ll _m = 0, ll _b = 0) {
      m = _m, b = _b;
    }
    ll eval(ll x) {
      return m * x + b;
    }
  };
  bool isBad(Line l1, Line l2, Line l3) {
    // bad when x of (l1 x l3) < x of (l1 x l2)
    return ((__int128)l3.b - l1.b) * ((__int128)l1.m -
        l2.m)
      <= ((__int128)l2.b - l1.b) * ((__int128)l1.m - l3
          .m);
  }
  vector<Line> lines;
  int pointer = 0;
  ConvexHullTrick() {
    lines.clear(), pointer = 0;
  }
  void reset() {
    lines.clear(), pointer = 0;
  }
  void add(ll m, ll b) { // insert line decreasing in
      slope
    while (!lines.empty() && lines.back().m == m &&
        lines.back().b > b)
      lines.pop_back();
    if (!lines.empty() && lines.back().m == m) return;
    Line newLine(m, b);
    while (lines.size() > 1 && isBad(lines[lines.size()
        - 2], lines.back(), newLine))
      lines.pop_back();
    lines.emplace_back(newLine);
  }
  ll query(ll x) { // return min
    if (lines.empty()) return INF;
    if (lines.size() == 1) return lines[0].eval(x);
    int low = 0, high = int(lines.size()) - 2, best =
        -1;
    while (low <= high) {
      int mid = (low + high) >> 1;
      if (lines[mid + 1].eval(x) < lines[mid].eval(x))
        low = (best = mid) + 1;
      else high = mid - 1;
    }
    return lines[best + 1].eval(x);
  }
  ll get(ll x) { //answer query increasing in x
    if (lines.empty()) return INF;
    if (pointer >= lines.size()) pointer = lines.size()
        - 1;
    while (pointer + 1 < lines.size() && lines[pointer +
         1].eval(x) < lines[pointer].eval(x))
      ++pointer;
    return lines[pointer].eval(x);
  }
} cht;
```

FenwickTree2d.h

Description: Computes sums a[i,j] for all 0<=i<=I, 0<=j<=J, and increases single elements a[i,j] (0-index).
Usage: Requires update positions to be known in advance in form vector<pair<int,int>> as constructor for FT2, or construct with limX only and call fakeUpdate().
Time: $\mathcal{O}\left(\log^2 N\right)$.
Status: stress-tested

47b6a8, 54 lines

```
struct FT {
  vector<ll> s;
  FT(int n) : s(n) {}
  void update(int pos, ll dif) { // a[pos] += dif
    for (; pos < sz(s); pos |= pos + 1) s[pos] += dif;
  }
  ll query(int pos) { // sum of values in [0, pos)
    ll res = 0;
    for (; pos; pos -= (pos & (-pos))) res += s[pos -
        1];
    return res;
  }
  int lower_bound(ll sum) {// min pos st sum of [0, pos]
      >= sum
    // Returns n if no sum is >= sum, or -1 if empty sum
        is.
    if (sum <= 0) return -1;
```

```
    int pos = 0;
    for (int pw = 1 << 25; pw; pw >>= 1) {
      if (pos + pw <= sz(s) && s[pos + pw - 1] < sum)
        pos += pw, sum -= s[pos - 1];
    }
    return pos;
  }
};
struct FT2 {
  vector<vi> ypos; vector<FT> ybit;
  bool call_init = false;
  void fakeUpdate(int x, int y) {
    for (; x < sz(ypos); x |= x + 1) ypos[x].push_back(y
        );
  }
  FT2(int limx, vector<pair<int,int>> &updates){
    ypos.resize(limx + 5);
    for(auto [x, y]: updates) fakeUpdate(x, y);
    init();
  }
  FT2(int limx) : ypos(limx + 5) {}
  void init() {
    call_init = true;
    ybit.clear();
    for (vi& v : ypos) sort(all(v)), ybit.emplace_back(
        sz(v));
  }
  int get_id(int x, int y) {
    return (int)(lower_bound(all(ypos[x]), y) - ypos[x].
        begin());
  }
  void update(int x, int y, ll dif) {
    if(!call_init) init();
    for (; x < sz(ypos); x |= x + 1) ybit[x].update(
        get_id(x, y), dif);
  }
  ll query(int x, int y) { //calculates sum for <=x, <=y
      .
    if(!call_init) init();
    x++;
    ll sum = 0;
    for (; x; x -= (x & (-x))) sum += ybit[x - 1].query(
        upper_bound(all(ypos[x - 1]), y) - ypos[x - 1].
        begin());
    return sum;
  }
};
```

ImplicitTreap.h

Description: An implicit treap on an array that supports deleting an element, inserting an element in any position and splitting the array into two parts (prefix/suffix).

Usage: Declare pnode t = nullptr for an empty array, then use the functions. If you want to handle more data in each node, then reimplement refresh(). Note that the array is 1-indexed.

Time: $\mathcal{O}(\log N)$

Status: Tested on one problem

27f8be, 39 lines

```
mt19937_64 rng(6736);
struct node{
  int val, sz = 1;
  uint64_t y;
  node *le = nullptr, *ri = nullptr;
  node(int val): val(val), y(rng()) {}
};
namespace implicit_treap{ //1-index
  using pnode = node*;
  int get_sz(pnode &p){ return p != nullptr ? p->sz : 0;
      }
  void refresh(pnode &p){ p->sz = get_sz(p->le) + get_sz
      (p->ri) + 1; }
  //first is [1, i], second is [i + 1, n]
  pair<pnode, pnode> split(pnode t, int i){
    if(t == nullptr) return {nullptr, nullptr};
    pair<pnode, pnode> ans;
    if(get_sz(t->le) + 1 <= i){
      ans = split(t->ri, i - get_sz(t->le) - 1);
      t->ri = ans.first; ans.first = t;
      refresh(ans.first);
    }
    else{
      ans = split(t->le, i);
      t->le = ans.second; ans.second = t;
      refresh(ans.second);
    }
    return ans;
  }
  //t1 is treap of a1,...,an. t2 is treap of b1,...,bm.
  //Merge into a1,...,an,b1,...,bm.
  pnode merge(pnode t1, pnode t2){
    if(t1 == nullptr || t2 == nullptr) return t1 ==
        nullptr ? t2 : t1;
    if(t1->y >= t2->y){
      t1->ri = merge(t1->ri, t2);
      refresh(t1); return t1;
    }
    t2->le = merge(t1, t2->le);
    refresh(t2); return t2;
  }
}
```

Matrix.h

Description: Matrix multiplication modulo MOD. Stored entries must be in [0, MOD).

Usage: Set MOD; 8 * MOD * MOD must fit in signed 64-bit.

Status: Randomized multiplication and exponentiation regression test

14a4ba, 24 lines

```
struct Matrix {
  static const ll MOD = 1000000007; // Set to the
      required modulus.
  static const uint64_t modmod = 8ULL * MOD * MOD; //
      must be < 2^63
  vector<vector<uint64_t>> val;
  int n, m;
  void resize(int _n, int _m) {
    n = _n, m = _m;
    val.assign(n, vector<uint64_t>(m));
  }
  Matrix operator * (const Matrix &other) {
    Matrix ret(n, other.m);
    for (int i = 0; i < n; ++i)
      for (int k = 0; k < m; ++k) {
        for (int j = 0; j < other.m; ++j)
          ret[i][j] += val[i][k] * other[k][j];
        if (k % 8 == 0) {
          for (int j = 0; j < other.m; ++j)
            ret[i][j] = min(ret[i][j], ret[i][j] -
                modmod);
        }
      }
    for (auto &row : ret.val) for (auto &x : row) x %=
        MOD;
    return ret;
  }
};
```

MoQueries.h

Description: Using Mo's Algorithm to answer tree path (vertex) or interval queries, adding/removing one element each step.

Usage: change block size and Type of ans in t_ans and blsz.
Define add, remove and calc in namespace MoAlgo after including, then call mo_inter() or mo_tree().
Note that the array/tree is 0-index.

Time: $\mathcal{O}\left(N\sqrt{Q}\right)$

Status: Tested on one tree problem and one interval problem.

4ea09c, 60 lines

```
namespace MoAlgo{
  using T_ans = ll;
  const int blsz = 350; // block size
  //(inter[0], inter[1]) is the current interval/path
      BEFORE OPERATION
  void add(int idx, array<int, 2> &inter); // add idx
  void remove(int idx, array<int, 2> &inter); // remove
      idx
  T_ans calc(array<int, 2> &inter);
  // compute current answer
  pii get_block(pii que){
    if((que.first / blsz) & 1) que.second = -que.second;
    return {que.first / blsz, que.second};
  }
  inline vector<T_ans> mo_tree(vector<pii> que, vector<
      vi>& graph, int root=0){
    int n = sz(graph);
```

```
    array<int, 2> path = {root, root};
    vi order(sz(que)), in(n), out(n), on_path(n), pa(n);
    vector<T_ans> ans(sz(que));
    int tim = 0;
    add(root, path), on_path[root] = 1;
    auto dfs = [&](int u, int p, auto& self) -> void {
      pa[u] = p;
      in[u] = tim; tim++;
      for (int v : graph[u]) if (v != p) self(v, u, self
          );
      out[u] = tim; tim++;
    };
    dfs(root, -1, dfs);
    iota(all(order), 0);
    auto cmp = [&](int s, int t){
      pii ps = {in[que[s].first], in[que[s].second]};
      pii pt = {in[que[t].first], in[que[t].second]};
      return get_block(ps) < get_block(pt);
    };
    sort(all(order), cmp);
    for(int qi: order){
      vi tmp;
      tmp.reserve(n);
      auto move_end = [&](int &a, int b){
        auto step = [&](int c){
          if (on_path[c]){
            remove(a, path); on_path[a] = 0;
          }
          else{
            add(c, path); on_path[c] = 1;
          }
          a = c;
        };
        for(;!(in[b] <= in[a] && in[a] <= out[b]); b =
            pa[b]){
          tmp.push_back(b);
        }
        while (a != b) step(pa[a]);
        for(int i = (int)tmp.size() - 1; i >= 0; i--)
            step(tmp[i]);
        tmp.clear(),tmp.resize(0);
      };
      move_end(path[0], que[qi].first);
      move_end(path[1], que[qi].second);
      ans[qi] = calc(path);
    }
    return ans;
  }
}
```

SegmentTreeLazyIterative.h

Description: Non-recursion segment tree with range updates and range queries.

Usage: `T, U are the node and lazy types. TT, UT, UU represents merging two nodes, applying lazy to a node, and merging two lazies.`

Time: $\mathcal{O}(\log N)$ for each operation

Status: Tested on range add update and range min query.

f55804, 82 lines

```
template<class T,class U> struct segtree{
  //fill merge node + node here
  T TT(T &le, T &ri);
  //fill apply lazy "app" variable to "node" variable
      here
  void UT(U &app, T &node);
  //Append new lazy "node" to existing lazy "app",
      assigning to "app"
  void UU(U &app, U &node);
  //Tid, Uid are default values of T, U,
  //representing empty node and empty lazy (no change).
  T Tid;
  U Uid;
  segtree(T _Tid, U _Uid, vector<T> &a){
    Tid = _Tid, Uid = _Uid;
    build(a);
  }
  vector<T> seg;
  vector<U> d;
  int n, lg;
  void refresh(int v){
    seg[v] = TT(seg[v<<1], seg[(v<<1)|1]);
  }
  //0-index
  void build(vector<T> &a){
    int siz = a.size();
    for(n = 1; n < siz;) n <<= 1;
    seg.assign(n<<1, Tid), d.assign(n, Uid), lg = __lg(n
        );
    for(int i = 0; i < siz; i++) seg[n + i] = a[i];
    for(int i = n - 1; i >= 1; i--) refresh(i);
  }
  void apply(int v, U &f){
    UT(f, seg[v]);
    if(v < n) UU(d[v], f);
  }
  void push(int v){
    apply(v << 1, d[v]), apply((v << 1) | 1, d[v]);
    d[v] = Uid;
  }
  void update(int l, int r, U &f){ //update in [l, r]
    l += n, r += (n + 1);
    if(l >= r) return;
    for(int i = lg; i >= 1; i--){
      if((l >> i << i) != l) push(l >> i);
      if((r >> i << i) != r) push((r - 1) >> i);
    }
    int l0 = l, r0 = r;
    for(;l < r; l >>= 1, r >>= 1){
      if(l & 1){
        apply(l, f);
        l++;
      }
      if(r & 1){
        r--;
        apply(r, f);
      }
    }
    l = l0, r = r0;
    for(int i = 1; i <= lg; i++){
      if((l >> i << i) != l) refresh(l >> i);
      if((r >> i << i) != r) refresh((r - 1) >> i);
    }
  }
  T query(int l, int r){ // query in [l, r]
    l += n, r += (n + 1);
    if(l >= r) return Tid;
    for(int i = lg; i >= 1; i--){
      if((l >> i << i) != l) push(l >> i);
      if((r >> i << i) != r) push((r - 1) >> i);
    }
    T le = Tid, ri = Tid;
    for(;l < r; l >>= 1, r >>= 1){
      if(l & 1){
        le = TT(le, seg[l]);
        l++;
      }
      if(r & 1){
        r--;
        ri = TT(seg[r], ri);
      }
    }
    return TT(le, ri);
  }
};
```

SplayTree.h

Description: Implicit splay tree with insertion, deletion and access by 1-based sequence position.

Time: $\mathcal{O}(\log N)$ amortized per operation

Status: Tested on SPOJ

dc3105, 101 lines

```
struct Node {
  Node *parent, *child[2];
  int sz = 0, value;
  Node(int val = 0) {
    parent = nullptr, sz = 1, value = val;
    child[0] = child[1] = nullptr;
}
  int side() {
    return parent ? parent->child[1] == this : -1;
  }
  void pushUp() {
    sz = 1;
    for (int i = 0; i < 2; ++i)
      if (child[i]) sz += child[i]->sz;
  }
};
const int NUM_NODES = 1e6 + 5;
Node nodes[NUM_NODES];
int numNode = 0;
```

```
Node *newNode(int value = 0) {
  assert(numNode < NUM_NODES);
  nodes[numNode] = Node(value);
  return &nodes[numNode++];
}
struct SplayTree {
  Node *root;
  SplayTree() {
    root = newNode(0);
    link(root, newNode(0), 1);
    root->pushUp();
  }
  Node *oneDir(int dir) {
    Node *p = root;
    while (p && p->child[dir]) p = p->child[dir];
    return p;
  }
  void link(Node *a, Node *b, int dir) {
    if (a == nullptr) {
      root = b;
      if (root) root->parent = nullptr;
      return;
    }
    a->child[dir] = b;
    if (b) b->parent = a;
  }
  void rotate(Node *a) {
    Node *p = a->parent;
    int dir = a->side();
    link(p->parent, a, p->side());
    link(p, a->child[!dir], dir);
    link(a, p, !dir);
    p->pushUp(), a->pushUp();
  }
  void splay(Node *a, Node *s = nullptr) {
    while (a->parent != s) {
      Node *p = a->parent;
      if (p->parent == s) rotate(a);
      else if (a->side() == p->side()) rotate(p), rotate
          (a); // zig-zig
      else rotate(a), rotate(a);
    }
  }
  // Zero-based rank including sentinels; target must be
       below s if s != nullptr.
  void find(int k, Node *s = nullptr) {
    assert(0 <= k && k < root->sz);
    Node *p = root;
    while (true) {
      int cntLeft = (p->child[0] ? p->child[0]->sz : 0);
      if (cntLeft > k) p = p->child[0];
      else if (cntLeft + 1 > k) {
        splay(p, s); return;
      }
      else {
        k -= cntLeft + 1;
        p = p->child[1];
      }
    }
  }
  // Insert before the current k-th element. 1 <= k <= n
       + 1
  void insert(int k, int val) {
    assert(1 <= k && k <= root->sz - 1);
    find(k), find(k - 1, root);
    Node *foo = newNode(val);
    link(foo, root->child[0], 0);
    link(root, foo, 0);
    foo->pushUp(), root->pushUp();
  }
  // 1 <= k <= n
  void remove(int k) {
    assert(1 <= k && k <= root->sz - 2);
    find(k + 1), find(k - 1, root);
    link(root->child[0], nullptr, 1);
    root->child[0]->pushUp();
    root->pushUp();
  }
  // 1 <= k <= n
  int get(int k) {
    assert(1 <= k && k <= root->sz - 2);
    find(k);
    return root->value;
  }
} tree;
```

TimeSegmentTree.h

Description: A time segment tree, used to support deletion in a DS that supports addition and rollback with an additional $\log N$ factor. Note that the queries from the DS has to be offline.

Usage: 1: Edit MAX_TIME, ope, Q (description below)
2: Define add(), rollback() and calc() in namespace segtree_time after including.
3: Turn the addition/deletion updates into tuples [x,l,r],
which means perform update x at time l and delete at time r + 1.
4: Call update(l, r, x) on all tuples, then call solve_queries()
to solve the queries offline.
rollback(k) to rollback k times, unite() and Findset() to merge and find root.

Time: $\mathcal{O}(T(n) \log N)$, where $T(n)$ is the time complexity for the DS.

Status: Tested on one problem

39cb7b, 55 lines

```
namespace segtree_time{ //time is 1-indexed
  //ope is the update type put into the segtree,
  //Q is the type of the query answers.
  using ope = pii;
  using Q = int;
  //MAX_TIME is the maximum time allowed
  const int MAX_TIME = 3e5;
  vector<vector<ope>> group(4 * MAX_TIME + 5);
  void update(int v, int TL, int TR, int l, int r, const
       ope &x){
    if(r < TL || l > TR) return;
    if(l <= TL && TR <= r){
      group[v].push_back(x); return;
    }
    int TM = ((TL + TR) >> 1);
    update(v << 1, TL, TM, l, r, x);
    update((v << 1) | 1, TM + 1, TR, l, r, x);
  }
  void update(int l, int r, const ope &x){
    assert(1 <= l && l <= r && r <= MAX_TIME);
    update(1, 1, MAX_TIME, l, r, x);
  }
  Q calc(int i); // Define to answer the i-th query.
  void add(ope &x); // Define to apply x to your DS.
  //rollback the last k operations of your DS here.
  //Note that ALL operations are counted, EVEN IF they
       don't change the DS.
  void rollback(int k);
  inline void DFS(int v, int TL, int TR, vector<pii> &
       que, int &ptr, vector<Q> &ans){
    if(ptr == que.size()) return;
    for(ope &x: group[v]) add(x);
    if(TL != TR){
      int TM=((TL + TR) >> 1);
      DFS(v << 1, TL, TM, que, ptr, ans);
      DFS((v << 1) | 1,TM + 1, TR, que, ptr, ans);
    }
    else{
      for(; ptr < que.size() && que[ptr].first == TL;
          ptr++){
        ans[que[ptr].second] = calc(que[ptr].second);
      }
    }
    rollback(group[v].size());
  }
  //a query is of format {time_of_query, index_of_query}
  inline vector<Q> solve_queries(vector<pii> &queries){
    for (auto [time, index] : queries) {
      assert(1 <= time && time <= MAX_TIME);
      assert(0 <= index && index < (int)queries.size());
    }
    sort(queries.begin(), queries.end());
    int ptr = 0;
    vector<Q> ans(queries.size());
    DFS(1, 1, MAX_TIME, queries, ptr, ans);
    return ans;
  }
};
using namespace segtree_time;
```

Treap.h

Description: An explicit treap that supports split(), merge(), insert(), and erase().

Usage: Declare pnode t = nullptr for an empty treap, then use the functions. If you want to handle more data in each node, then reimplement get_val() and refresh().
Time: $\mathcal{O}(\log N)$
Status: Tested on two problems

f08209, 37 lines

```
mt19937_64 rng(6736);
struct node{
    int x, sz = 1;
    ull y;
    node *le = nullptr, *ri = nullptr;
    node(int x): x(x), y(rng()) {}
};
namespace treap{
    using pnode = node*;
    int get_val(pnode &p){ return p != nullptr ? p->sz :
        0; }
    void refresh(pnode &p){ p->sz = get_val(p->le) +
        get_val(p->ri) + 1; }
    //first is <=x, second is >x
    pair<pnode, pnode> split(pnode t, int x){
        if(t == nullptr) return {nullptr, nullptr};
        pair<pnode, pnode> ans;
        if(t->x <= x){
            ans = split(t->ri, x);
            t->ri = ans.fi; ans.fi = t;
            refresh(ans.fi);
        } else {
            ans = split(t->le, x);
            t->le = ans.se; ans.se = t;
            refresh(ans.se);
        }
        return ans;
    }
    //assume all t1->x < t2->x (ordering between trees)
    pnode merge(pnode t1, pnode t2){
        if(t1 == nullptr || t2 == nullptr) return t1 ==
            nullptr ? t2 : t1;
        if(t1->y >= t2->y){
            t1->ri = merge(t1->ri, t2);
            refresh(t1); return t1;
        }
        t2->le = merge(t1, t2->le);
        refresh(t2); return t2;
    }
}
```

Graph (7)

7.1 Graph theory

7.1.1 Number of Spanning Trees

Create an $N \times N$ matrix mat, and for each edge $a \to b \in G$, do mat[a][b]--, mat[b][b]++ (and mat[b][a]--, mat[a][a]++ if G is undirected). Remove the i-th row and column and take the determinant; this yields the number of directed spanning trees rooted at i (if G is undirected, remove any row/column).

7.1.2 Erdős–Gallai theorem

A simple graph with node degrees $d_1 \geq \cdots \geq d_n$ exists iff $d_1 + \cdots + d_n$ is even and for every $k = 1 \ldots n$,

$$\sum_{i=1}^{k} d_i \leq k(k-1) + \sum_{i=k+1}^{n} \min(d_i, k).$$

7.2 Algorithms

TwoSat.h
Description: 2sat -> finding an arbitrary equation
Time: $\mathcal{O}(n+m)$
Status: Tested by KickingKun

485abb, 58 lines

```
struct TwoSat { // convert vector -> array to get higher
    speed ....
  int n; vector <int> lab, num, low; vector <vector <int
      >> adj;
  int timeDfs, scc; stack <int> st;
  TwoSat (int _n) {
    this->n = _n; lab.assign(n * 2 + 1, 0); low.assign(n
        * 2 + 1, 0);
    num.assign(n * 2 + 1, 0); adj.assign(n * 2 + 1,
        vector <int>());
    timeDfs = scc = 0; while (!st.empty()) st.pop();
  }
  int NON(int u) {
    return u <= n ? u + n : u - n;
  }
  void add(int u, int v) {
    adj[u].emplace_back(v);
  }
  void add_u_or_v(int u, int v) { // u or v = 1
    adj[NON(u)].emplace_back(v); adj[NON(v)].
        emplace_back(u);
  }
  void assign(int u, bool val) { // u = val
    if (val == 1) add_u_or_v(u, u);
    else add_u_or_v(NON(u), NON(u));
  }
  void add_equal(int u, int v) { // u = v
    add_u_or_v(u, NON(v)); add_u_or_v(NON(u), v);
  }
  void add_non_equal(int u, int v) { // u != v
    add_u_or_v(u, v); add_u_or_v(NON(u), NON(v));
  }
  void dfs(int u) {
    low[u] = num[u] = ++timeDfs; st.emplace(u);
    for (int v: adj[u]) {
      if (lab[v]) continue;
      if (!num[v]) dfs(v), low[u] = min(low[u], low[v]);
      else low[u] = min(low[u], num[v]);
    }
    if (low[u] == num[u]) {
      ++scc;
      while (true) {
        int v = st.top(); st.pop();
        lab[v] = scc; if (v == u) break;
      }
    }
  }
  bool check_exist() {
    for (int i = 1; i <= n; i++) {
      if (!lab[NON(i)]) dfs(NON(i));
      if (!lab[i]) dfs(i);
    }
    for (int i = 1; i <= n; i++)
      if (lab[i] == lab[NON(i)]) return false;
    return true;
  }
  vector <int> get_equation() {
    if (!check_exist()) return {-1};
    vector <int> ans = {0};
    for (int i = 1; i <= n; i++) ans.emplace_back(lab[i]
        < lab[NON(i)]);
    return ans;
  }
};
```

BipartieMatching.h
Description: use for max matching
Time: $\mathcal{O}\left(E \cdot \sqrt{V}\right)$
Status: approved by KickingKun

b7e4ae, 74 lines

```
struct bipartite_matching {
  const int n, m;
  vector <int> match1, match2, que, dist;
  vector <vector<int>> adj;
  bipartite_matching(int _n, int _m) : n(_n), m(_m) {
    match1.assign(n + 1, -1); match2.assign(m + 1, -1);
    que.assign(n + 1, 0); dist.assign(n + 1, 0); adj.
        assign(n + 1, vector <int>());
  }
  void add(int u, int v) {
    adj[u].emplace_back(v);
  }
  bool bfs() {
    fill(dist.begin(), dist.end(), -1);
    int queBegin = 1, queEnd = 1;
    for (int u = 1; u <= n; ++u) {
```

```
    if (match1[u] == -1) dist[u] = 0, que[queEnd++] =
        u;
  }
  bool success = false;
  while (queBegin < queEnd) {
    int u = que[queBegin++];
    for (int v : adj[u]) {
      if (match2[v] == -1) {
        success = true;
      }
      else if (dist[match2[v]] == -1) {
        dist[match2[v]] = dist[u] + 1;
        que[queEnd++] = match2[v];
      }
    }
  }
  return success;
}
bool dfs(int u) {
  for (int v : adj[u]) {
    if (match2[v] == -1 || (dist[match2[v]] == dist[u]
         + 1 && dfs(match2[v]))) {
      match1[u] = v; match2[v] = u; dist[u] = n + m;
      return true;
    }
  }
  dist[u] = n + m;
  return false;
}
int max_matching() {
  while (bfs()) {
    for (int u = 1; u <= n; ++u) {
      if (match1[u] == -1) dfs(u);
    }
  }
  return n - count(match1.begin() + 1, match1.end(),
      -1);
}
pair <vector <int>, vector <int>> minimum_vertex_cover
    () {
  vector <int> L, R;
  for (int u = 1; u <= n; ++u) {
    if (dist[u] == -1) L.emplace_back(u);
    else if (match1[u] != -1) R.emplace_back(match1[u
        ]);
  }
  return {L, R};
}
pair <vector <int>, vector <int>>
    maximum_independent_set() {
  auto [_L, _R] = minimum_vertex_cover();
  vector <int> L, R;
  vector <bool> mark1(n + 2), mark2(m + 2);
  for (int x: _L) mark1[x] = true; for (int x: _R)
      mark2[x] = true;
  for (int u = 1; u <= n; u++) if (!mark1[u]) L.
      emplace_back(u);
  for (int u = 1; u <= m; u++) if (!mark2[u]) R.
      emplace_back(u);
  return {L, R};
}
vector <pii> get_edges() { // get from max matching
  vector <pii> ans;
  for (int u = 1; u <= n; ++u)
    if (match1[u] != -1) ans.emplace_back(u, match1[u
        ]);
  return ans;
}
};
```

MaxFlow.h

Description: Dinitz's algorithm to find max-flow.
Usage: `Call myflow.resize(n) to initalize the network.`
Time: $\mathcal{O}\left(V^2 \cdot E\right)$ in worst cases
Status: Well-tested

3be25f, 64 lines

```
struct Dinic {
  struct Edge {
    int u, v;
    ll capacity, flow = 0;
    Edge(int _u = 0, int _v = 0, ll _capacity = 0) {
      u = _u, v = _v, capacity = _capacity, flow = 0;
    }
  };
  const ll FLOW_INF = (ll)2e18;
  vector<Edge> edges;
  vector<vector<int>> adj;
  vector<int> level, ptr;
  int n, m;
  Dinic(int n = 0) { resize(n); }
  void resize(int _n) {
    n = _n, m = 0;
    adj.assign(n + 1, vector<int>()), edges.clear();
  }
  int add(int u, int v, ll capacity) {
    edges.emplace_back(u, v, capacity);
    edges.emplace_back(v, u, 0);
    adj[u].emplace_back(m++), adj[v].emplace_back(m++);
    return m - 2;
  }
  ll dfs(int u, int sink, ll pushed) {
    if (u == sink || pushed == 0) return pushed;
    for (int &i = ptr[u]; i < (int)adj[u].size(); ++i) {
      int id = adj[u][i]; Edge e = edges[id];
      if (level[u] + 1 != level[e.v] || e.capacity - e.
          flow <= 0) continue;
      if (ll p = dfs(e.v, sink, min(pushed, e.capacity -
           e.flow))) {
        edges[id].flow += p, edges[id ^ 1].flow -= p;
        return p;
      }
    }
    return 0;
  }
  ll maxFlow(int source, int sink) {
    assert(source != sink);
    ll flow = 0;
    do {
      level = ptr = vector<int>(n + 1, 0);
      queue<int> q; q.emplace(source); level[source] =
          1;
      while (!q.empty()) {
        int u = q.front(); q.pop();
        for (auto id : adj[u]) {
          if (level[edges[id].v] || edges[id].capacity -
               edges[id].flow <= 0) continue;
          level[edges[id].v] = level[u] + 1;
          q.emplace(edges[id].v);
        }
      }
      while (ll pushed = dfs(source, sink, FLOW_INF))
          flow += pushed;
    } while (level[sink] != 0);
    return flow;
  }
  vector<pii> minCut() { // remember to call maxFlow(
      source, sink) and be careful of duplicates
    vector<pii> cut;
    for (int id = 0; id < m; id += 2) {
      auto &e = edges[id];
      if (e.capacity > 0 && level[e.u] && !level[e.v])
        cut.emplace_back(e.u, e.v);
    }
    return cut;
  }
} myflow;
```

MinCostMaxFlow.h

Description: Edmond-Karp algorithm with SPFA to find the shortest path.
Usage: `Call myflow.resize(n) to initalize the network.`
Time: $\mathcal{O}\left(F \cdot V \cdot E\right)$ in worst cases
Status: Well-tested

357dcc, 65 lines

```
struct MCMF {
  struct Edge {
    int u, v;
    ll capacity, cost, flow = 0;
    Edge(int _u = 0, int _v = 0, ll _capacity = 0, ll
        _cost = 0) {
      u = _u, v = _v, capacity = _capacity, cost = _cost
          , flow = 0;
    }
  };
  const ll FLOW_INF = (ll)2e18;
  vector<Edge> edges;
  vector<vector<int>> adj;
  vector<int> level, ptr;
  int n, m;
```

```
MCMF(int n = 0) { resize(n); }
void resize(int _n) {
  n = _n, m = 0;
  adj.assign(n + 1, vector<int>()), edges.clear();
}
int add(int u, int v, ll capacity, ll cost) {
  edges.emplace_back(u, v, capacity, cost);
  edges.emplace_back(v, u, 0, -cost);
  adj[u].emplace_back(m++), adj[v].emplace_back(m++);
  return m - 2;
}
bool spfa(int source, int sink, vector<ll> &dist,
    vector<int> &par) {
  dist.assign(n + 1, FLOW_INF), par.assign(n + 1, -1);
  vector<bool> inQueue(n + 1, false); queue<int> q;
  dist[source] = 0; inQueue[source] = true; q.emplace(
      source);
  while (!q.empty()) {
    int u = q.front(); q.pop();
    inQueue[u] = false;
    for (auto id : adj[u]) {
      int v = edges[id].v;
      if (edges[id].capacity - edges[id].flow <= 0)
          continue;
      if (dist[v] > dist[u] + edges[id].cost) {
        dist[v] = dist[u] + edges[id].cost;
        par[v] = id;
        if (!inQueue[v]) {
          inQueue[v] = true;
          q.emplace(v);
        }
      }
    }
  }
  return dist[sink] != FLOW_INF;
}
pair<ll, ll> minCost(int source, int sink) {
  assert(source != sink);
  ll flow = 0, cost = 0; vector<ll> dist; vector<int>
      par;
  while (spfa(source, sink, dist, par)) {
    ll amount = FLOW_INF - flow; int cur = sink;
    while (cur != source) {
      amount = min(amount, edges[par[cur]].capacity -
          edges[par[cur]].flow);
      cur = edges[par[cur]].u;
    }
    flow += amount, cost += amount * dist[sink], cur =
        sink;
    while (cur != source) {
      edges[par[cur]].flow += amount;
      edges[par[cur] ^ 1].flow -= amount;
      cur = edges[par[cur]].u;
    }
  }
  return make_pair(flow, cost);
  }
} myflow;
```

BiconnectedComponents.h

Description: Finds all biconnected components in an undirected graph, and runs a callback for the edges in each. In a biconnected component there are at least two internally disjoint paths between any two nodes (a cycle exists through them). Note that a node can be in several components. An edge which is not in a component is a bridge, i.e., not part of any cycle.

Usage: `int eid = 0; ed.resize(N);`
`for each edge (a,b) {`
`ed[a].emplace_back(b, eid);`
`ed[b].emplace_back(a, eid++); }`
`bicomps([&](const vi& edgelist) {...});`

Time: $\mathcal{O}(E+V)$

Status: tested during MIPT ICPC Workshop 2017

c6b7c7, 31 lines

```
vi num, st;
vector<vector<pii>> ed;
int Time;
template<class F>
int dfs(int at, int par, F& f) {
  int me = num[at] = ++Time, top = me;
  for (auto [y, e] : ed[at]) if (e != par) {
    if (num[y]) {
      top = min(top, num[y]);
      if (num[y] < me)
        st.push_back(e);
    } else {
      int si = sz(st);
      int up = dfs(y, e, f);
      top = min(top, up);
      if (up == me) {
        st.push_back(e);
        f(vi(st.begin() + si, st.end()));
        st.resize(si);
      }
      else if (up < me) st.push_back(e);
      else { /* e is a bridge */ }
    }
  }
  return top;
}
template<class F>
void bicomps(F f) {
  num.assign(sz(ed), 0);
  rep(i,0,sz(ed)) if (!num[i]) dfs(i, -1, f);
}
```

DFSMatching.h

Description: Simple bipartite matching algorithm. Graph g should be a list of neighbors of the left partition, and *btoa* should be a vector full of -1's of the same size as the right partition. Returns the size of the matching. *btoa*[i] will be the match for vertex i on the right side, or -1 if it's not matched.

Usage: `vi btoa(m, -1); dfsMatching(g, btoa);`

Time: $\mathcal{O}(VE)$

Status: works

522b98, 22 lines

```
bool find(int j, vector<vi>& g, vi& btoa, vi& vis) {
  if (btoa[j] == -1) return 1;
  vis[j] = 1; int di = btoa[j];
  for (int e : g[di])
    if (!vis[e] && find(e, g, btoa, vis)) {
      btoa[e] = di;
      return 1;
    }
  return 0;
}
int dfsMatching(vector<vi>& g, vi& btoa) {
  vi vis;
  rep(i,0,sz(g)) {
    vis.assign(sz(btoa), 0);
    for (int j : g[i])
      if (find(j, g, btoa, vis)) {
        btoa[j] = i;
        break;
      }
  }
  return sz(btoa) - (int)count(all(btoa), -1);
}
```

EdgeColoring.h

Description: Given a simple, undirected graph with max degree D, computes a $(D+1)$-coloring of the edges such that no neighboring edges share a color. (D-coloring is NP-hard, but can be done for bipartite graphs by repeated matchings of max-degree nodes.)

Time: $\mathcal{O}(NM)$

Status: stress-tested, tested on kattis:gamescheduling

e210e2, 31 lines

```
vi edgeColoring(int N, vector<pii> eds) {
  vi cc(N + 1), ret(sz(eds)), fan(N), free(N), loc;
  for (pii e : eds) ++cc[e.first], ++cc[e.second];
  int u, v, ncols = *max_element(all(cc)) + 1;
  vector<vi> adj(N, vi(ncols, -1));
  for (pii e : eds) {
    tie(u, v) = e;
    fan[0] = v;
    loc.assign(ncols, 0);
    int at = u, end = u, d, c = free[u], ind = 0, i = 0;
    while (d = free[v], !loc[d] && (v = adj[u][d]) !=
        -1)
      loc[d] = ++ind, cc[ind] = d, fan[ind] = v;
    cc[loc[d]] = c;
    for (int cd = d; at != -1; cd ^= c ^ d, at = adj[at
        ][cd])
      swap(adj[at][cd], adj[end = at][cd ^ c ^ d]);
    while (adj[fan[i]][d] != -1) {
      int left = fan[i], right = fan[++i], e = cc[i];
      adj[u][e] = left;
      adj[left][e] = u;
      adj[right][e] = -1;
      free[right] = e;
```

```
    }
    adj[u][d] = fan[i];
    adj[fan[i]][d] = u;
    for (int y : {fan[0], u, end})
      for (int& z = free[y] = 0; adj[y][z] != -1; z++);
  }
  rep(i,0,sz(eds))
    for (tie(u, v) = eds[i]; adj[u][ret[i]] != v;) ++ret
        [i];
  return ret;
}
```

EulerWalk.h

Description: Eulerian undirected/directed path/cycle algorithm. Input should be a vector of (dest, global edge index), where for undirected graphs, forward/backward edges have the same index. Returns a list of nodes in the Eulerian path/cycle with src at both start and end, or empty list if no cycle/path exists. To get edge indices back, add .second to s and ret.
Time: $\mathcal{O}(V+E)$
Status: stress-tested

780b64, 15 lines

```
vi eulerWalk(vector<vector<pii>>& gr, int nedges, int
    src=0) {
  int n = sz(gr);
  vi D(n), its(n), eu(nedges), ret, s = {src};
  D[src]++; // to allow Euler paths, not just cycles
  while (!s.empty()) {
    int x = s.back(), y, e, &it = its[x], end = sz(gr[x
        ]);
    if (it == end){ ret.push_back(x); s.pop_back();
        continue; }
    tie(y, e) = gr[x][it++];
    if (!eu[e]) {
      D[x]--, D[y]++;
      eu[e] = 1; s.push_back(y);
    }}
  for (int x : D) if (x < 0 || sz(ret) != nedges+1)
      return {};
  return {ret.rbegin(), ret.rend()};
}
```

LinkCutTree.h

Description: Represents a forest of unrooted trees. You can add and remove edges (as long as the result is still a forest), and check whether two nodes are in the same tree.
Time: All operations take amortized $\mathcal{O}(\log N)$.
Status: Stress-tested a bit for N <= 20

0fb462, 88 lines

```
struct Node { // Splay tree. Root's pp contains tree's
    parent.
  Node *p = 0, *pp = 0, *c[2];
  bool flip = 0;
  Node() { c[0] = c[1] = 0; fix(); }
  void fix() {
    if (c[0]) c[0]->p = this;
    if (c[1]) c[1]->p = this;
    // (+ update sum of subtree elements etc. if wanted)
  }
  void pushFlip() {
    if (!flip) return;
    flip = 0; swap(c[0], c[1]);
    if (c[0]) c[0]->flip ^= 1;
    if (c[1]) c[1]->flip ^= 1;
  }
  int up() { return p ? p->c[1] == this : -1; }
  void rot(int i, int b) {
    int h = i ^ b;
    Node *x = c[i], *y = b == 2 ? x : x->c[h], *z = b ?
        y : x;
    if ((y->p = p)) p->c[up()] = y;
    c[i] = z->c[i ^ 1];
    if (b < 2) {
      x->c[h] = y->c[h ^ 1];
      y->c[h ^ 1] = x;
    }
    z->c[i ^ 1] = this;
    fix(); x->fix(); y->fix();
    if (p) p->fix();
    swap(pp, y->pp);
  }
  void splay() {
    for (pushFlip(); p; ) {
      if (p->p) p->p->pushFlip();
      p->pushFlip(); pushFlip();
      int c1 = up(), c2 = p->up();
      if (c2 == -1) p->rot(c1, 2);
      else p->p->rot(c2, c1 != c2);
    }
  }
  Node* first() {
    pushFlip();
    return c[0] ? c[0]->first() : (splay(), this);
  }
};
struct LinkCut {
  vector<Node> node;
  LinkCut(int N) : node(N) {}
  void link(int u, int v) { // add an edge (u, v)
    assert(!connected(u, v));
    makeRoot(&node[u]);
    node[u].pp = &node[v];
  }
  void cut(int u, int v) { // remove an edge (u, v)
    Node *x = &node[u], *top = &node[v];
    makeRoot(top); x->splay();
    assert(top == (x->pp ?: x->c[0]));
    if (x->pp) x->pp = 0;
    else {
      x->c[0] = top->p = 0;
      x->fix();
    }
  }
  bool connected(int u, int v) { // are u, v in the same
      tree?
    Node* nu = access(&node[u])->first();
    return nu == access(&node[v])->first();
  }
  void makeRoot(Node* u) {
    access(u);
    u->splay();
    if(u->c[0]) {
      u->c[0]->p = 0;
      u->c[0]->flip ^= 1;
      u->c[0]->pp = u;
      u->c[0] = 0;
      u->fix();
    }
  }
  Node* access(Node* u) {
    u->splay();
    while (Node* pp = u->pp) {
      pp->splay(); u->pp = 0;
      if (pp->c[1]) {
        pp->c[1]->p = 0; pp->c[1]->pp = pp; }
      pp->c[1] = u; pp->fix(); u = pp;
    }
    return u;
  }
};
```

SCC.h

Description: Finds strongly connected components in a directed graph. If vertices u, v belong to the same component, we can reach u from v and vice versa.
Usage: scc(graph, [&](vi& v) { ... }) visits all components in reverse topological order. comp[i] holds the component index of a node (a component only has edges to components with lower index). ncomps will contain the number of components.
Time: $\mathcal{O}(E+V)$
Status: Bruteforce-tested for N <= 5

76b5c9, 23 lines

```
vi val, comp, z, cont;
int Time, ncomps;
template<class G, class F> int dfs(int j, G& g, F& f) {
  int low = val[j] = ++Time, x; z.push_back(j);
  for (auto e : g[j]) if (comp[e] < 0)
    low = min(low, val[e] ?: dfs(e,g,f));
  if (low == val[j]) {
    do {
      x = z.back(); z.pop_back();
      comp[x] = ncomps;
      cont.push_back(x);
    } while (x != j);
    f(cont); cont.clear();
    ncomps++;
```

```
  }
  return val[j] = low;
}
template<class G, class F> void scc(G& g, F f) {
  int n = sz(g);
  val.assign(n, 0); comp.assign(n, -1);
  Time = ncomps = 0;
  rep(i,0,n) if (comp[i] < 0) dfs(i, g, f);
}
```

WeightedMatching.h

Description: Given a weighted bipartite graph, matches every node on the left with a node on the right such that no nodes are in two matchings and the sum of the edge weights is minimal. Takes cost[N][M], where cost[i][j] = cost for L[i] to be matched with R[j] and returns (min cost, match), where L[i] is matched with R[match[i]]. Negate costs for max cost. Requires $N \leq M$.

Time: $\mathcal{O}\left(N^2M\right)$

Status: Tested on kattis:cordonbleu, stress-tested

1e0fe9, 31 lines

```
pair<int, vi> hungarian(const vector<vi> &a) {
  if (a.empty()) return {0, {}};
  int n = sz(a) + 1, m = sz(a[0]) + 1;
  vi u(n), v(m), p(m), ans(n - 1);
  rep(i,1,n) {
    p[0] = i;
    int j0 = 0; // add "dummy" worker 0
    vi dist(m, INT_MAX), pre(m, -1);
    vector<bool> done(m + 1);
    do { // dijkstra
      done[j0] = true;
      int i0 = p[j0], j1, delta = INT_MAX;
      rep(j,1,m) if (!done[j]) {
        auto cur = a[i0 - 1][j - 1] - u[i0] - v[j];
        if (cur < dist[j]) dist[j] = cur, pre[j] = j0;
        if (dist[j] < delta) delta = dist[j], j1 = j;
      }
      rep(j,0,m) {
        if (done[j]) u[p[j]] += delta, v[j] -= delta;
        else dist[j] -= delta;
      }
      j0 = j1;
    } while (p[j0]);
    while (j0) { // update alternating path
      int j1 = pre[j0];
      p[j0] = p[j1], j0 = j1;
    }
  }
  rep(j,1,m) if (p[j]) ans[p[j] - 1] = j - 1;
  return {-v[0], ans}; // min cost
}
```

Geometry (8)

8.1 Point/Vector

TemplatePoint.h

Description: Point/Vector representation. geo_T is the type of the point (double or int or ll). Note that division doesn't work when geo_T = ll (floor division);

6a3dff, 28 lines

```
// FILE: Point.h
using geo_T = double;
struct pt{
  geo_T x, y;
  pt (geo_T x=0, geo_T y=0) : x(x), y(y) {}
  pt operator+(pt p){ return pt(x + p.x, y + p.y);}
  pt operator-(pt p){ return pt(x - p.x, y - p.y);}
  pt operator*(geo_T c){ return pt(x * c, y * c);}
  friend pt operator*(geo_T c, pt p){ return pt(p.x * c,
       p.y * c);}
  pt operator*(pt p){
    return pt(x * p.x - y * p.y, x * p.y + y * p.x);
  }
  pt operator/(geo_T c){ return pt(x / c, y / c);}
  bool operator==(pt p){ return x == p.x && y == p.y;}
};
int sign(geo_T x){ return (x < 0 ? -1 : (x == 0 ? 0 : 1)
    );}
geo_T sq_len(pt p){ return p.x * p.x + p.y * p.y;}
double len(pt p){ return sqrtl(sq_len(p));}
geo_T dot(pt a, pt b){ return a.x * b.x + a.y * b.y;}
geo_T cross(pt a, pt b){ return a.x * b.y - a.y * b.x;}
//rotates a vector v by 90 degrees ccw.
pt perp(pt v){ return pt(-v.y, v.x);}
//applies the linear transformation (combination of
    translation, rotation, scaling)
//that moves line p-q to fp-fq on point r.
pt linearTransform(pt p, pt q, pt r, pt fp, pt fq) {
  pt pq = q - p, pr = r - p, fpq = fq - fp;
  return fp + (dot(pq, pr) * fpq + cross(pq, pr) * perp(
      fpq)) / sq_len(pq);
}
```

TemplateAngle.h

Description: A class for ordering angles (represented by a point and a number of rotations around the origin). Useful for rotational sweeping. Note that points are ordered in counterclockwise order. Works with geo_T = ll;

"TemplatePoint.h" d44450, 14 lines

```
//assumes x^2 + y^2 > 0
struct angle{
  pt p; int t;
  angle(pt p, int t=0): p(p), t(t) {}
  angle(geo_T x, geo_T y, int t=0): p(x, y), t(t) {}
  int half(){ return p.y < 0 || (p.y == 0 && p.x < 0);}
  angle turn_90(){ return angle(perp(p), t + (half() &&
      p.x >= 0));}
  angle turn_180(){ return angle(-p.x, -p.y, t + half())
      ; }
  angle turn_360(){ return angle(p, t + 1);}
};
bool operator<(angle a, angle b){
  return make_tuple(a.t, a.half(), 0) <
    make_tuple(b.t, b.half(), cross(a.p, b.p));
}
```

8.2 Line/Segment

TemplateLine.h

Description: Line representation and common line functions. line is represented by a vector v and a number c, denoting all points p such that $v \times p = c$. Note that v is parallel to the line. Works for geo_T = ll except for line_proj and line_intersect

"TemplatePoint.h" 6e1ded, 18 lines

```
//assume v is nonzero.
struct line{
  pt v; geo_T c;
  line(pt a, pt b): v(b - a), c(cross(v, a)) {}
  geo_T side(pt p){ return cross(v, p) - c;}
};
//get the shortest distance from p to a point in line l.
double line_dist(pt p, line l) {return abs(l.side(p)) /
    len(l.v);}
//get the perpendicular line of l through point p
line perp_line(pt p, line l) { return line(p, p + perp(l
    .v));}
//get line projection, set refl=true to get line
    reflection instead
pt line_proj(pt p, line l, bool refl=false){
  return p - perp(l.v) * l.side(p) * (1 + refl) / sq_len
      (l.v);
}
pt line_intersect(line l1, line l2){
  assert(cross(l1.v, l2.v) != 0); //assume non-parallel
      lines
  return (l2.v * l1.c - l1.v * l2.c) / cross(l1.v, l2.v)
      ;
}
```

TemplateSegment.h

Description: Common segment functions. Each segment is represented by a pair of points. Works for geo_T = ll except for seg_intersect

"TemplatePoint.h", "TemplateLine.h" 97f11e, 24 lines

```
//check whether p is on segment s
bool on_seg(pt p, pair<pt, pt> s){
  return cross(p - s.fi, s.se - s.fi) == 0 && dot(p - s.
      fi, p - s.se) <= 0;
}
//returns shortest distance from p to a point in s
double seg_dist(pt p, pair<pt, pt> s) {
  pt v = s.se - s.fi;
  auto cmpProj = [&](pt &a, pt &b){ return dot(v, a) <=
      dot(v, b);};
  if(s.fi != s.se && cmpProj(s.fi, p) && cmpProj(p, s.se
      ))
    return line_dist(p, line(s.fi, s.se));
  return min(len(p - s.fi), len(p - s.se));
```

```
}
vector<pt> seg_intersect(pair<pt, pt> s1, pair<pt, pt>
    s2){
  pt a = s1.fi, b = s1.se, c = s2.fi, d = s2.se;
  line l1 = line(a, b), l2 = line(c, d);
  if(!same_side(a, b, l2) && !same_side(c, d, l1))
    return {line_intersect(l1, l2)};
  vector<pt> v;
  for(pt p: {a, b}) if(on_seg(p, s2)) v.push_back(p);
  for(pt p: {c, d}) if(on_seg(p, s1)) v.push_back(p);
  auto cmp = [&](pt &a, pt &b){ return tie(a.x, a.y) <
      tie(b.x, b.y);};
  sort(v.begin(), v.end(), cmp); v.erase(unique(v.begin
      (), v.end()), v.end());
  return v;
}
```

8.3 Circle

TemplateCircle.h

Description: Common circle geometry functions. Requires geo_T=double.

"TemplatePoint.h", "TemplateLine.h" 8a67ef, 44 lines

```
vector<pt> circle_circle_inter(pt o1, double r1, pt o2,
    double r2){
  double d_sq = sq_len(o2 - o1);
  if(d_sq == 0) return {}; //assume different circles
  double mido1_mul_d = (d_sq + r1 * r1 - r2 * r2) / 2;
  double h_sq = r1 * r1 - mido1_mul_d * mido1_mul_d /
      d_sq;
  vector<pt> ans;
  if(h_sq >= 0){
    pt mid = o1 + (o2 - o1) * mido1_mul_d / d_sq;
    pt h = perp(o2 - o1) * sqrt(h_sq / d_sq);
    ans = {mid - h, mid + h};
    if (h_sq == 0) ans.pop_back();
  }
  return ans;
}
vector<pt> circle_line_inter(pt o, double r, line l){
  double h_sq = r * r - line_dist(o, l) * line_dist(o, l
      );
  vector<pt> ans;
  if(h_sq >= 0){
    pt mid = line_proj(o, l), h = l.v * sqrt(h_sq) / len
        (l.v);
    ans = {mid - h, mid + h};
    if(h_sq == 0) ans.pop_back();
  }
  return ans;
}
//if r2 > 0, gets the outer tangents of circle (o1, r1)
    and (o2, r2).
//if r2 < 0, gets the inner tangent of circle (o1, r1)
    and (o2, -r2)
//set r2 = 0 to get tangents through a point.
vector<pair<pt, pt>> tangents(pt o1, double r1, pt o2,
    double r2) {
  double dr = r1 - r2, d_sq = sq_len(o2 - o1);
  double h_sq = d_sq - dr * dr;
  if(d_sq == 0 || h_sq < 0) return {}; //assume
      different circles
  vector<pair<pt, pt>> ans;
  for (double sign : {-1, 1}) {
    pt dir = ((o2 - o1) * dr + perp(o2 - o1) * sqrt(h_sq
        ) * sign) / d_sq;
    ans.push_back({o1 + dir * r1, o2 + dir * r2});
  }
  if (h_sq == 0) ans.pop_back();
  return ans;
}
//finds the center of the circle going through points a,
    b, c.
pt circumCenter(pt a, pt b, pt c) {
  b = b-a, c = c-a; assert(cross(b,c) != 0); //assume
      points not collinear.
  return a + perp(b * sq_len(c) - c * sq_len(b)) / (
      cross(b, c) * 2);
}
```

MinimumEnclosingCircle.h

Description: Computes the minimum circle that encloses a set of points. The code has lots of precision problems, so please use long double instead of double.

Time: expected $\mathcal{O}(n)$

Status: Tested on library checker (with long double instead).

"TemplatePoint.h", "TemplateCircle.h" 239c08, 19 lines

```
const double EPS = 1e-10;
mt19937 rng(6736);
pair<pt, double> min_enclosing_circle(vector<pt> v){
  shuffle(v.begin(), v.end(), rng);
  pt o = v[0]; double r = 0;
  for(int i = 0; i < v.size(); i++){
    if(r * r - sq_len(v[i] - o) >= -EPS) continue;
    o = v[i], r = 0;
    for(int j = 0; j < i; j++){
      if(r * r - sq_len(v[j] - o) >= -EPS) continue;
      o = (v[i] + v[j]) / 2.0; r = len(v[i] - o);
      for(int k = 0; k < j; k++){
        if(r * r - sq_len(v[k] - o) >= -EPS) continue;
        o = circumCenter(v[i], v[j], v[k]); r = len(v[i]
            - o);
      }
    }
  }
  return {o, r};
}
```

8.4 Polygon

TemplatePolygon.h

Description: Common polygon functions. Works when geo_T=ll except for polygon_center.

"TemplatePoint.h", "TemplateSegment.h" 1ad69a, 31 lines

```
double signed_polygon_area(vector<pt> &v){
  if(v.size() < 3) return 0;
  geo_T ans = 0;
  for(int i = 1; i + 1 < v.size(); i++)
    ans += cross(v[i] - v[0], v[i + 1] - v[0]);
  return ans / 2.0;
}
double polygon_area(vector<pt> &v){
  return abs(signed_polygon_area(v));
}
//check whether polygon is in ccw order
bool check_ccw(vector<pt> &v){return signed_polygon_area
    (v) >= 0;}
//check whether a is in polygon. turns strict=true to
    reject if a is in boundary.
bool in_polygon(vector<pt> &p, pt a, bool strict=false){
  int cnt = 0, n = p.size();
  for(int i = 0; i < n; i++){
    pt q = p[(i + 1) % n];
    if (on_seg(a, make_pair(p[i], q))) return !strict;
    if(((a.y < p[i].y) - (a.y < q.y)) * cross(p[i] - a,
        q - a) > 0) cnt ^= 1;
  }
  return cnt;
}
//returns the center of mass of a polygon
pt polygon_center(vector<pt>& v) {
  pt res(0, 0); geo_T A = 0;
  for (int i = 0, j = (int)v.size() - 1; i < (int)v.size
      (); j = i++) {
    geo_T cr = cross(v[j], v[i]);
    res = res + (v[i] + v[j]) * cr; A += cr;
  }
  return res / A / 3;
}
```

ConvexHull.h

Description: Returns the convex hull given set of points v in ccw order. Works when geo_T = ll;

Time: $\mathcal{O}(n \log n)$

"TemplatePoint.h" d63f26, 20 lines

```
vector<pt> convex_hull(vector<pt> v){
  if(v.size() <= 1) return v;
  auto cmp = [&](pt &a, pt &b){ return tie(a.x, a.y) <
      tie(b.x, b.y);};
  sort(v.begin(), v.end(), cmp);
  vector<pt> st; st.reserve(v.size() + 5);
  int s = 0;
  for(int _ = 0; _ < 2; _++){
    for(pt p: v){
```

```
    while(st.size() >= s + 2){
      pt a = st[st.size() - 2];
      if(cross(st.back() - a, p - a) <= 0) st.pop_back
          ();
      else break;
    }
    st.push_back(p);
  }
  reverse(v.begin(), v.end()); st.pop_back(); s = st.
      size();
 }
 if(st.size() == 2 && st[0] == st[1]) st.pop_back();
 return st;
}
```

PointInsideHull.h

Description: Determine whether a point t lies inside a convex polygon with no collinear points. The polygon can be in ccw or cw order. If strict is true, points on the boundary aren't included. If v.size() >= 3, then no need to include TemplateSegment.h
Time: $\mathcal{O}(\log N)$
Status: Tested on Atcoder

"TemplatePoint.h", "TemplateSegment.h" bfbb13, 13 lines

```
bool in_convex_poly(vector<pt> &v, pt p, bool strict=
    false){
 int low = 1, high = v.size() - 1, r = !strict, n = v.
     size();
 if(n <= 2) return r && on_seg(p, {v[0], v[1 % n]});
 if(cross(v[1] - v[0], v[n - 1] - v[0]) < 0) swap(low,
     high);
 if(sign(cross(p - v[0], v[high] - v[0])) <= -r) return
      false;
 if(sign(cross(p - v[0], v[low] - v[0])) >= r) return
     false;
 while(abs(high - low) > 1){
   int mid = ((low + high) >> 1);
   if(sign(cross(v[mid] - v[0], p - v[0])) == 1) low =
       mid;
   else high = mid;
 }
 return (cross(v[high] - v[low], p - v[low]) > -r);
}
```

HullDiameter.h

Description: Returns the two points with max distance on a polygon. If polygon is convex then no need to include ConvexHull.h.
Time: $\mathcal{O}(n)$

"TemplatePoint.h", "ConvexHull.h" b2be45, 22 lines

```
pair<pt, pt> hull_diameter(vector<pt> v, bool convex=
    true){
 if(!convex) v = convex_hull(v);
 int n = v.size(), ptr = 0; pair<pt, pt> ans = {v[0], v
     [0]};
 if(n == 1) return {v[0], v[0]};
 else if(n == 2) return {v[0], v[1]};
 auto upd = [&](pt &x, pt &y){
   if(sq_len(x - y) > sq_len(ans.fi - ans.se)) ans = {x
       , y};
 };
 for(int i = 0; i < n; i++){
   pt dir = v[(i + 1) % n] - v[i];
   for(;;ptr = (ptr + 1) % n){
     int p1 = sign(cross(dir, v[ptr] - v[(ptr + n - 1)
         % n]));
     int p2 = sign(cross(dir, v[(ptr + 1) % n] - v[ptr
         ]));
     if(ptr != i && ptr != (i + 1) % n && p1 != p2)
         break;
   }
   upd(v[i], v[ptr]), upd(v[(i + 1) % n], v[ptr]);
   if(cross(dir, v[(ptr + 1) % n]) == 0){
     upd(v[i], v[(ptr + 1) % n]), upd(v[(i + 1) % n], v
         [(ptr + 1) % n]);
   }
 }
 return ans;
}
```

LinePolygonIntersection.h

Description: Returns all segments of the intersection between a simple polygon and a line. The points must be given in ccw order.
Time: $\mathcal{O}(n \log n)$

"TemplatePoint.h", "TemplateLine.h" e4241d, 27 lines

```
vector<pair<pt, pt>> line_poly_inter(vector<pt> &v, line
    l){
 vector<pt> inter; vector<pair<pt, pt>> ans; int n = v.
     size();
 int prev = sign(l.side(v[n - 1])), curr = sign(l.side(
     v[0]));
 if(v.size() <= 2 && !l.side(v[0]) && !l.side(v[1]))
     ans.push_back({v[0], v[1]});
 for(int i = 0; i < n; i++){
   int nxt = sign(l.side(v[(i + 1) % n]));
   if(curr){
     if(nxt == -curr)
       inter.push_back(line_intersect(l, line(v[i], v[(
           i + 1) % n])));
   }
   else if(prev && nxt){
     inter.push_back(v[i]);
     if(prev == nxt) inter.push_back(v[i]);
   }
   else if(!prev && nxt){
     if(sign(dot(l.v, v[i] - v[(i + n - 1) % n])) ==
         nxt) inter.push_back(v[i]);
   }
   else if(prev && !nxt){
     if(sign(dot(l.v, v[(i + 1) % n] - v[i])) == prev)
         inter.push_back(v[i]);
   }
   prev = curr, curr = nxt;
 }
 auto cmp = [&](pt &a, pt &b){ return dot(l.v, a) < dot
     (l.v, b);};
 sort(inter.begin(), inter.end(), cmp);
 for(int i = 0; i < inter.size(); i += 2) ans.push_back
     ({inter[i], inter[i + 1]});
 return ans;
}
```

CirclePolygonIntersection.h

Description: Returns the area of the intersection of a circle with a ccw polygon.
Time: $\mathcal{O}(n)$
Status: Tested on one problem, stress-tested.

"TemplatePoint.h" 638723, 18 lines

```
double circlePoly(pt c, double r, vector<pt> ps) {
 auto arg = [](pt p, pt q) { return atan2(cross(p, q),
     dot(p, q)); };
 auto tri = [&](pt p, pt q) {
   auto r2 = r * r / 2;
   pt d = q - p;
   auto d2 = sq_len(d), a = dot(d, p) / d2, b = (sq_len
       (p) - r * r) / d2;
   auto det = a * a - b;
   if (det <= 0) return arg(p, q) * r2;
   auto s = max(0., -a-sqrt(det)), t = min(1., -a+sqrt(
       det));
   if (t < 0 || 1 <= s) return arg(p, q) * r2;
   pt u = p + d * s, v = p + d * t;
   return arg(p, u) * r2 + cross(u, v) / 2 + arg(v, q)
       * r2;
 };
 auto sum = 0.0;
 for (int i = 0; i < ps.size(); ++i)
   sum += tri(ps[i] - c, ps[(i + 1) % ps.size()] - c);
 return sum;
}
```

8.5 Sweep line Algorithms

ClosestPair.h

Description: Finds the closest pair of points. In order to do so, sort points by x, sweep line. Denote current min as d. Check all the points in rectangle $x-d, y-d$; $x, y+d$, there are at most 8 points.
Time: $\mathcal{O}(n \log n)$

ManhattanMST.h

Description: Given N points, returns up to 4*N edges, which are guaranteed to contain a minimum spanning tree for the graph with edge weights w(p, q) = $|p.x - q.x| + |p.y - q.y|$. Edges are returned in pair of indices.
Time: $\mathcal{O}(N \log N)$
Status: Tested on Library Checker

"TemplatePoint.h" f3bbce, 21 lines

```
vector<pair<int, int>> manhattan_mst(vector<pt> v){
 vector<pair<int, int>> ans; vector<int> order(v.size()
     );
```

```
  iota(order.begin(), order.end(), 0);
  for(int rot = 0; rot < 4; rot++){
    set<pair<int, int>> se;
    auto cmp = [&](int i, int j){ return v[i].x + v[i].y
        < v[j].x + v[j].y;};
    sort(order.begin(), order.end(), cmp);
    for(int idx: order){
      auto it = se.upper_bound({v[idx].y, 1e9 + 7});
      while(it != se.begin()){
        auto last = prev(it); int idx2 = last->se;
        if(v[idx2].x - v[idx2].y > v[idx].x - v[idx].y)
            break;
        ans.push_back({idx, idx2}); se.erase(last);
      }
      se.insert({v[idx].y, idx});
    }
    if(rot & 1) for(int i = 0; i < v.size(); i++) v[i].x
        = -v[i].x;
    else for(int i = 0; i < v.size(); i++) swap(v[i].x,
        v[i].y);
  }
  return ans;
}
```

Strings (9)

AhoCorasick.h

Description: Aho-corasick.
Usage: `Call init() before any operations.`
Time: $\mathcal{O}(len(s))$ for addition, $\mathcal{O}(total_len)$ for build(), $\mathcal{O}(len(x))$ or $\mathcal{O}(numofoccurences)$ for query()
Status: Tested on some problems

9db8f2, 59 lines

```
struct AhoCorasick {
  static const int SIZE = 26;
  static const char START_CHAR = 'a';
  struct Node {
    int fail, link;
    int child[SIZE], nxt[SIZE];
    int output;
    Node() {
      fail = 0, link = -1, output = 0;
      memset(child, -1, sizeof child);
      memset(nxt, -1, sizeof nxt);
    }
  };
  vector<Node> nodes;
  void init() {
    nodes.clear(), nodes.emplace_back(Node());
  }
  void add(string &s, int id) {
    int v = 0;
    for (char ch : s) {
      int c = ch - START_CHAR;
      if (nodes[v].child[c] == -1) {
        nodes[v].child[c] = nodes.size();
        nodes.emplace_back(Node());
      }
      v = nodes[v].child[c];
    }
    ++nodes[v].output;
  }
  void build() {
    nodes[0].fail = nodes[0].link = 0;
    for (int i = 0; i < SIZE; ++i)
      nodes[0].nxt[i] = (nodes[0].child[i] == -1 ? 0 :
          nodes[0].child[i]);
    queue<int> q; q.emplace(0);
    while (!q.empty()) {
      int u = q.front(); q.pop();
      for (int c = 0; c < SIZE; ++c) if (nodes[u].child[
          c] != -1) {
        int v = nodes[u].child[c];
        nodes[v].fail = (u == 0 ? 0 : nodes[nodes[u].
            fail].nxt[c]);
        for (int d = 0; d < SIZE; ++d)
          nodes[v].nxt[d] = (nodes[v].child[d] == -1 ?
              nodes[nodes[v].fail].nxt[d] : nodes[v].
              child[d]);
        if (nodes[nodes[v].fail].output != 0) nodes[v].
            link = nodes[v].fail;
        else nodes[v].link = nodes[nodes[v].fail].link;
        q.emplace(v);
      }
    }
  }
  int query(string &x) {
    int ans = 0, p = 0;
    for (auto ch : x) {
      int c = ch - START_CHAR, nxt = nodes[p].nxt[c];
      for (int u = nxt; u != 0; u = nodes[u].link) {
        ans += nodes[u].output;
      }
      p = nxt;
    }
    return ans;
  }
} aho;
```

SuffixArray.h

Description: Suffix array using prefix doubling to sort with Kasai's algorithm for LCP
Time: $\mathcal{O}(N \log^2 N)$ for construcing suffix array and $\mathcal{O}(n)$ for construcing LCP array
Status: Tested on judge.yosupo.jp

981801, 48 lines

```
struct SuffixArray {
  string str;
  vector<int> sa, lcp, order, rank;
  vector<ll> weight;
  int n;
  SuffixArray(string _str = "") {
    str = _str, n = str.size();
    sa.assign(n, 0), lcp.assign(n, 0), order.assign(n,
        0);
    rank.assign(n, 0), weight.assign(n, 0);
    getSA(), getLCP();
  }
  void getSA() {
    for (int i = 0; i < n; ++i) {
      sa[i] = i;
      weight[i] = (unsigned char)str[i];
    }
    auto cmp = [&](int x, int y) {
      return weight[x] < weight[y];
    };
    sort (sa.begin(), sa.end(), cmp);
    for (int len = 1; len <= n; len <<= 1) {
      int cnt = 0;
      for (int i = 0; i < n; ++i) {
        if (i > 0 && weight[sa[i]] == weight[sa[i - 1]])
            order[sa[i]] = order[sa[i - 1]];
        else order[sa[i]] = ++cnt;
      }
      if (cnt == n) break;
      for (int i = 0; i < n; ++i) {
        weight[i] = 1ll * order[i] * (n + 1) + (i + len
            < n ? order[i + len] : 0);
      }
      for (int i = 0, j = 0; i < n; i = j) {
        while (j < n && order[sa[j]] == order[sa[i]]) ++
            j;
        sort (sa.begin() + i, sa.begin() + j, cmp);
      }
    }
  }
  void getLCP() {
    for (int i = 0; i < n; ++i) rank[sa[i]] = i;
    for (int i = 0, k = 0; i < n; lcp[rank[i++]] = k) {
      if (rank[i] > 0) {
        int j = sa[rank[i] - 1];
        if (k > 0) --k;
        while (i + k < n && j + k < n && str[i + k] ==
            str[j + k]) ++k;
      }
      else k = 0;
    }
  }
} T;
```

KMP.h

Description: pi[x] computes the length of the longest prefix of s that ends at x, other than s[0...x] itself (abacaba -> 0010123). Can be used to find all occurrences of a string.
Time: $\mathcal{O}(n)$
Status: Tested on kattis:stringmatching

d4375c, 15 lines

```
vi pi(const string& s) {
  vi p(sz(s));
  rep(i,1,sz(s)) {
```

```
    int g = p[i-1];
    while (g && s[i] != s[g]) g = p[g-1];
    p[i] = g + (s[i] == s[g]);
  }
  return p;
}
vi match(const string& s, const string& pat) {
  vi p = pi(pat + '\0' + s), res;
  rep(i,sz(p)-sz(s),sz(p))
    if (p[i] == sz(pat)) res.push_back(i - 2 * sz(pat));
  return res;
}
```

Manacher.h

Description: For each position in a string, computes p[0][i] = half length of longest even palindrome around pos i, p[1][i] = longest odd (half rounded down).
Time: $\mathcal{O}(N)$
Status: Stress-tested

e7ad79, 13 lines

```
array<vi, 2> manacher(const string& s) {
  int n = sz(s);
  array<vi,2> p = {vi(n+1), vi(n)};
  rep(z,0,2) for (int i=0,l=0,r=0; i < n; i++) {
    int t = r-i+!z;
    if (i<r) p[z][i] = min(t, p[z][l+t]);
    int L = i-p[z][i], R = i+p[z][i]-!z;
    while (L>=1 && R+1<n && s[L-1] == s[R+1])
      p[z][i]++, L--, R++;
    if (R>r) l=L, r=R;
  }
  return p;
}
```

MinRotation.h

Description: Finds the lexicographically smallest rotation of a string.
Usage: `rotate(v.begin(), v.begin()+minRotation(v), v.end());`
Time: $\mathcal{O}(N)$
Status: Stress-tested

d07a42, 8 lines

```
int minRotation(string s) {
  int a=0, N=sz(s); s += s;
  rep(b,0,N) rep(k,0,N) {
    if (a+k == b || s[a+k] < s[b+k]) {b += max(0, k-1);
        break;}
    if (s[a+k] > s[b+k]) { a = b; break; }
  }
  return a;
}
```

PalindromicTree.h

Description: Builds palindromic tree for a string.
Status: stress-tested

f8784d, 22 lines

```
namespace PalindromicTree {
const int N = 100000 + 5, sigma = 26;
int s[N], len[N], link[N], to[N][sigma], n, last, sz;
void init() {
  fill(len, len + sz, 0); fill(link, link + sz, 0);
  for (int i = 0; i < sz; ++i) fill(to[i], to[i] + sigma
      , 0);
  n = last = 0; s[n++] = -1; link[0] = 1; len[1] = -1;
      sz = 2;
}
int get_link(int v) {
  while (s[n - len[v] - 2] != s[n - 1]) v = link[v];
  return v;
}
void add_letter(int c) {
  s[n++] = c; last = get_link(last);
  if (!to[last][c]) {
    len[sz] = len[last] + 2;
    link[sz] = to[get_link(link[last])][c];
    to[last][c] = sz++;
  }
  last = to[last][c];
}
}
```

SuffixAutomaton.h

Description: Suffix Automaton, supporting multiple strings. The beginning iterator is 0. Make calls to itmove to progress iterator.
Time: $\mathcal{O}(n)$
Status: stress-tested

be95c5, 29 lines

```
struct SuffixAutomaton{
  struct Node{
    int len, link = -1, nxt[26];
    Node(int len): len(len) {fill(nxt, nxt + 26, -1);}
    Node(int len, int link, int _nxt[26]):
    len(len), link(link) {memcpy(nxt, _nxt, sizeof nxt);
        }
  };
  vector<Node> g = {0};
  int ptrans(int p, char c, int nxt) {
    for (int q = g[p].nxt[c]; p != -1 && g[p].nxt[c] ==
        q;
      p = g[p].link) g[p].nxt[c] = nxt;
    return p;
  }
  int fbuild(int p, char c) {
    if (p == -1) return 0;
    int q = g[p].nxt[c];
    if (g[p].len + 1 == g[q].len) return q;
    int qc = g.size();
    g.emplace_back(g[p].len+1, g[q].link, g[q].nxt);
    ptrans(p, c, qc); return g[q].link = qc;
  }
  int addc(int p, char c) { // Progress iterator; c is
      in [0, 26).
    if (g[p].nxt[c] != -1) return fbuild(p, c);
    int nc = g.size();
    g.emplace_back(g[p].len+1);
    int r = fbuild(ptrans(p, c, nc), c);
    g[nc].link = r; return nc;
  }
};
```

Various (10)

FastMod.h

Description: Compute $a\%b$ about 5 times faster than usual, where b is constant but not known at compile time. Returns a value congruent to a (mod b) in the range $[0, 2b)$.
Status: proven correct, stress-tested

62d9a9, 7 lines

```
struct FastMod {
  uint64_t b, m;
  FastMod(uint64_t b) : b(b), m(-1ULL / b) {}
  uint64_t reduce(uint64_t a) { // a % b + (0 or b)
    return a - (uint64_t)((__uint128_t(m) * a) >> 64) *
        b;
  }
};
```

ConstantIntervals.h

Description: Split a monotone function on [from, to) into a minimal set of half-open intervals on which it has the same value. Runs a callback g for each such interval.
Usage: `constantIntervals(0, sz(v), [&](int x){return v[x];}, [&](int lo, int hi, T val){...});`
Time: $\mathcal{O}\left(k \log \frac{n}{k}\right)$
Status: tested

753a4c, 19 lines

```
template<class F, class G, class T>
void rec(int from, int to, F& f, G& g, int& i, T& p, T q
    ) {
  if (p == q) return;
  if (from == to) {
    g(i, to, p);
    i = to; p = q;
  } else {
    int mid = (from + to) >> 1;
    rec(from, mid, f, g, i, p, f(mid));
    rec(mid+1, to, f, g, i, p, q);
  }
}
template<class F, class G>
void constantIntervals(int from, int to, F f, G g) {
  if (to <= from) return;
  int i = from; auto p = f(i), q = f(to-1);
  rec(from, to-1, f, g, i, p, q);
  g(i, to, q);
}
```

DivideAndConquerDP.h

Description: Given $a[i] = \min_{lo(i) \le k < hi(i)}(f(i,k))$ where the (minimal) optimal k increases with i, computes $a[i]$ for $i = L..R-1$.
Time: $\mathcal{O}((N + (hi - lo)) \log N)$

Status: tested on http://codeforces.com/contest/321/problem/E
d38d2b, 17 lines

```
struct DP { // Modify at will:
  int lo(int ind) { return 0; }
  int hi(int ind) { return ind; }
  ll f(int ind, int k) { return dp[ind][k]; }
  void store(int ind, int k, ll v) { res[ind] = pii(k, v
      ); }
  void rec(int L, int R, int LO, int HI) {
    if (L >= R) return;
    int mid = (L + R) >> 1;
    pair<ll, int> best(LLONG_MAX, LO);
    rep(k, max(LO,lo(mid)), min(HI,hi(mid)))
      best = min(best, make_pair(f(mid, k), k));
    store(mid, best.second, best.first);
    rec(L, mid, LO, best.second+1);
    rec(mid+1, R, best.second, HI);
  }
  void solve(int L, int R) { rec(L, R, INT_MIN, INT_MAX)
      ; }
};
```

FastInput.h

Description: Read an integer from stdin. Usage requires your program to pipe in input from file.
Usage: `./a.out < input.txt`
Time: About 5x as fast as cin/scanf.
Status: tested on SPOJ INTEST, unit tested
7b3c70, 16 lines

```
inline char gc() { // like getchar()
  static char buf[1 << 16];
  static size_t bc, be;
  if (bc >= be) {
    buf[0] = 0, bc = 0;
    be = fread(buf, 1, sizeof(buf), stdin);
  }
  return buf[bc++]; // returns 0 on EOF
}
int readInt() {
  int a, c;
  while ((a = gc()) < 40);
  if (a == '-') return -readInt();
  while ((c = gc()) >= 48) a = a * 10 + c - 480;
  return a - 48;
}
```

FastKnapsack.h

Description: Given N non-negative integer weights w and a non-negative target t, computes the maximum S <= t such that S is the sum of some subset of the weights.
Time: $\mathcal{O}(N \max(w_i))$
Status: Tested on kattis:eavesdropperevasion, stress-tested
b20ccc, 16 lines

```
int knapsack(vi w, int t) {
  int a = 0, b = 0, x;
  while (b < sz(w) && a + w[b] <= t) a += w[b++];
  if (b == sz(w)) return a;
  int m = *max_element(all(w));
  vi u, v(2*m, -1);
  v[a+m-t] = b;
  rep(i,b,sz(w)) {
    u = v;
    rep(x,0,m) v[x+w[i]] = max(v[x+w[i]], u[x]);
    for (x = 2*m; --x > m;) rep(j, max(0,u[x]), v[x])
      v[x-w[j]] = max(v[x-w[j]], j);
  }
  for (a = t; v[a+m-t] < 0; a--) ;
  return a;
}
```

IntervalContainer.h

Description: Add and remove intervals from a set of disjoint intervals. Will merge the added interval with any overlapping intervals in the set when adding. Intervals are [inclusive, exclusive).
Time: $\mathcal{O}(\log N)$
Status: stress-tested
edce47, 22 lines

```
set<pii>::iterator addInterval(set<pii>& is, int L, int
    R) {
  if (L == R) return is.end();
  auto it = is.lower_bound({L, R}), before = it;
  while (it != is.end() && it->first <= R) {
    R = max(R, it->second);
    before = it = is.erase(it);
  }
  if (it != is.begin() && (--it)->second >= L) {
    L = min(L, it->first);
    R = max(R, it->second);
    is.erase(it);
  }
  return is.insert(before, {L,R});
}
void removeInterval(set<pii>& is, int L, int R) {
  if (L == R) return;
  auto it = addInterval(is, L, R);
  auto r2 = it->second;
  if (it->first == L) is.erase(it);
  else (int&)it->second = L;
  if (R != r2) is.emplace(R, r2);
}
```

IntervalCover.h

Description: Compute indices of smallest set of intervals covering another interval. Intervals should be [inclusive, exclusive). To support [inclusive, inclusive], change (A) to add `|| R.empty()`. Returns empty set on failure (or if G is empty).
Time: $\mathcal{O}(N \log N)$
Status: Tested on kattis:intervalcover
9e9d8d, 19 lines

```
template<class T>
vi cover(pair<T, T> G, vector<pair<T, T>> I) {
  vi S(sz(I)), R;
  iota(all(S), 0);
  sort(all(S), [&](int a, int b) { return I[a] < I[b]; }
      );
  T cur = G.first;
  int at = 0;
  while (cur < G.second) { // (A)
    pair<T, int> mx = make_pair(cur, -1);
    while (at < sz(I) && I[S[at]].first <= cur) {
      mx = max(mx, make_pair(I[S[at]].second, S[at]));
      at++;
    }
    if (mx.second == -1) return {};
    cur = mx.first;
    R.push_back(mx.second);
  }
  return R;
}
```

KnuthDP.h

Description: When doing DP on intervals: $a[i][j] = \min_{i<k<j}(a[i][k] + a[k][j]) + f(i,j)$, where the (minimal) optimal k increases with both i and j, one can solve intervals in increasing order of length, and search $k = p[i][j]$ for $a[i][j]$ only between $p[i][j-1]$ and $p[i+1][j]$. This is known as Knuth DP. Sufficient criteria for this are if $f(b,c) \le f(a,d)$ and $f(a,c) + f(b,d) \le f(a,d) + f(b,c)$ for all $a \le b \le c \le d$. Consider also: LineContainer (ch. Data structures), monotone queues, ternary search.
Time: $\mathcal{O}\left(N^2\right)$